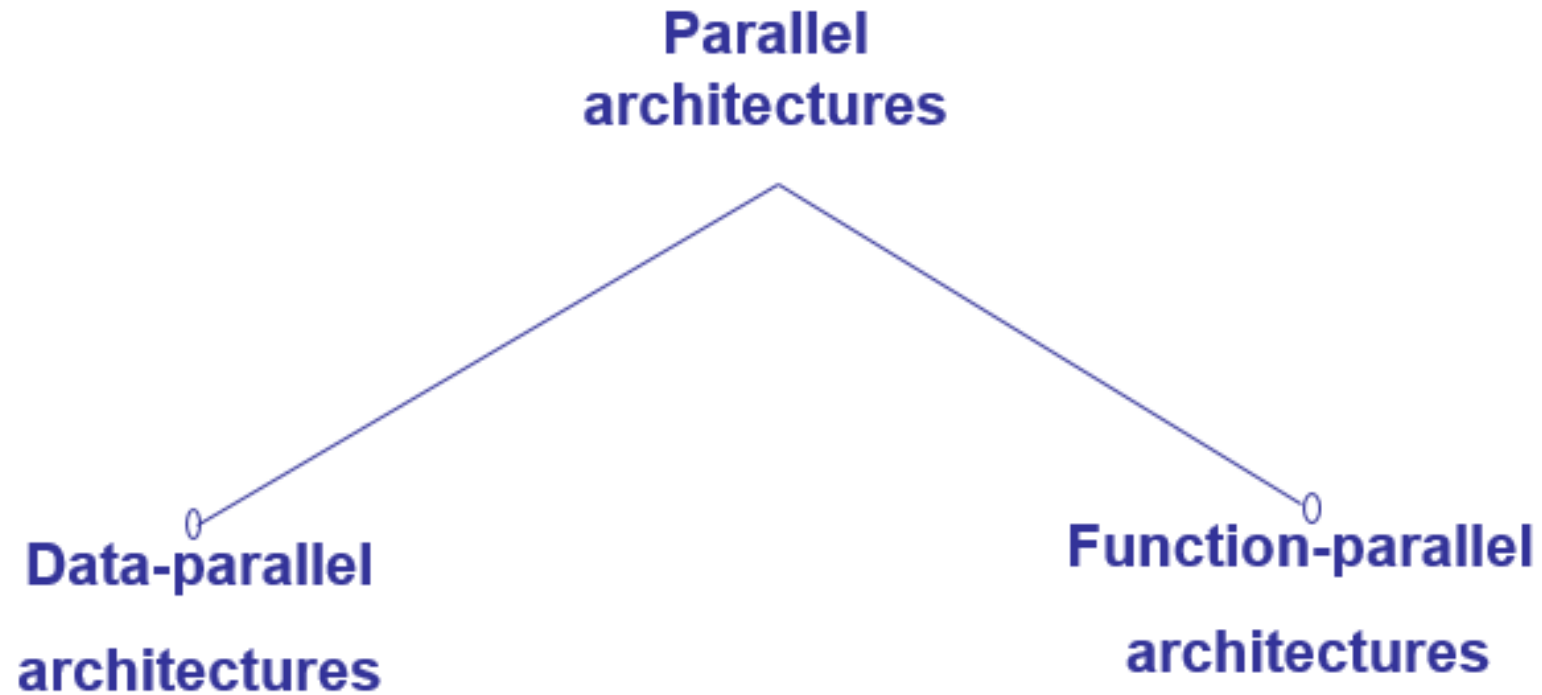
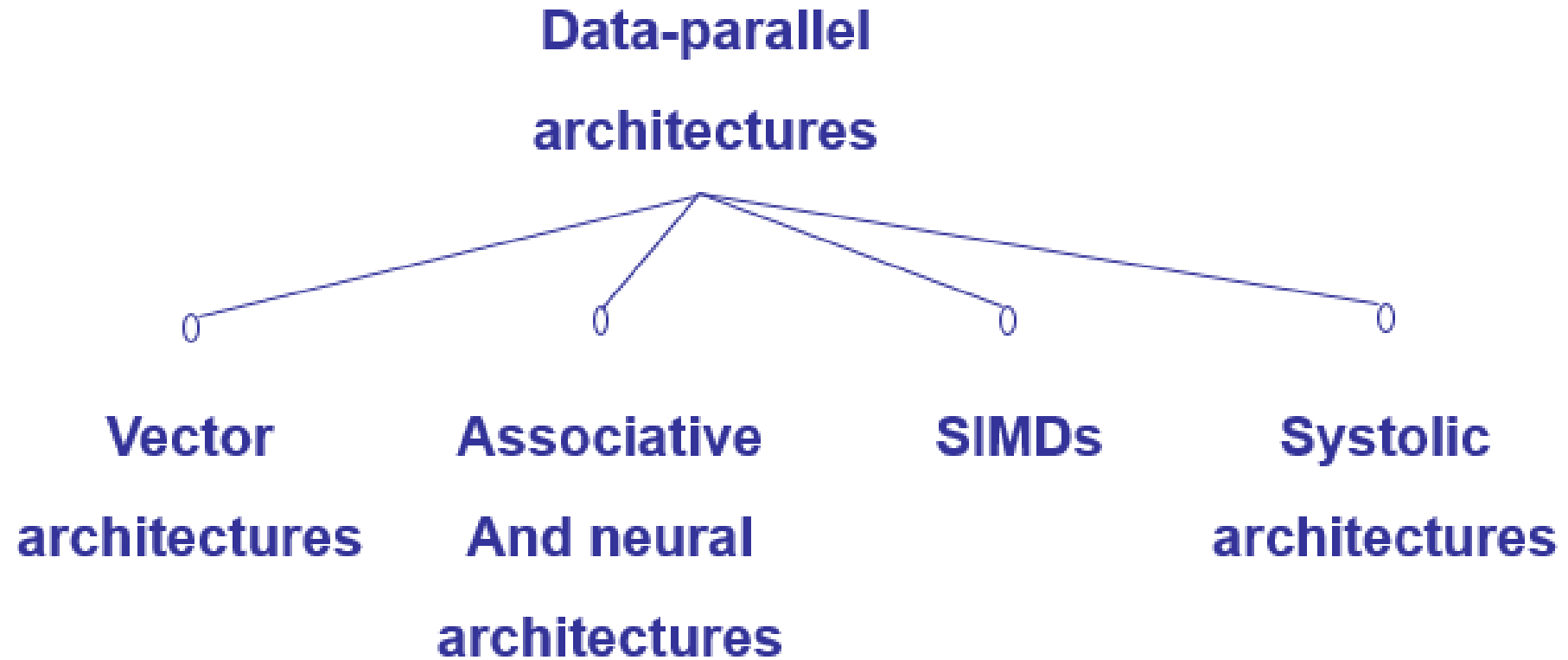


PARALLEL ARCHITECTURES

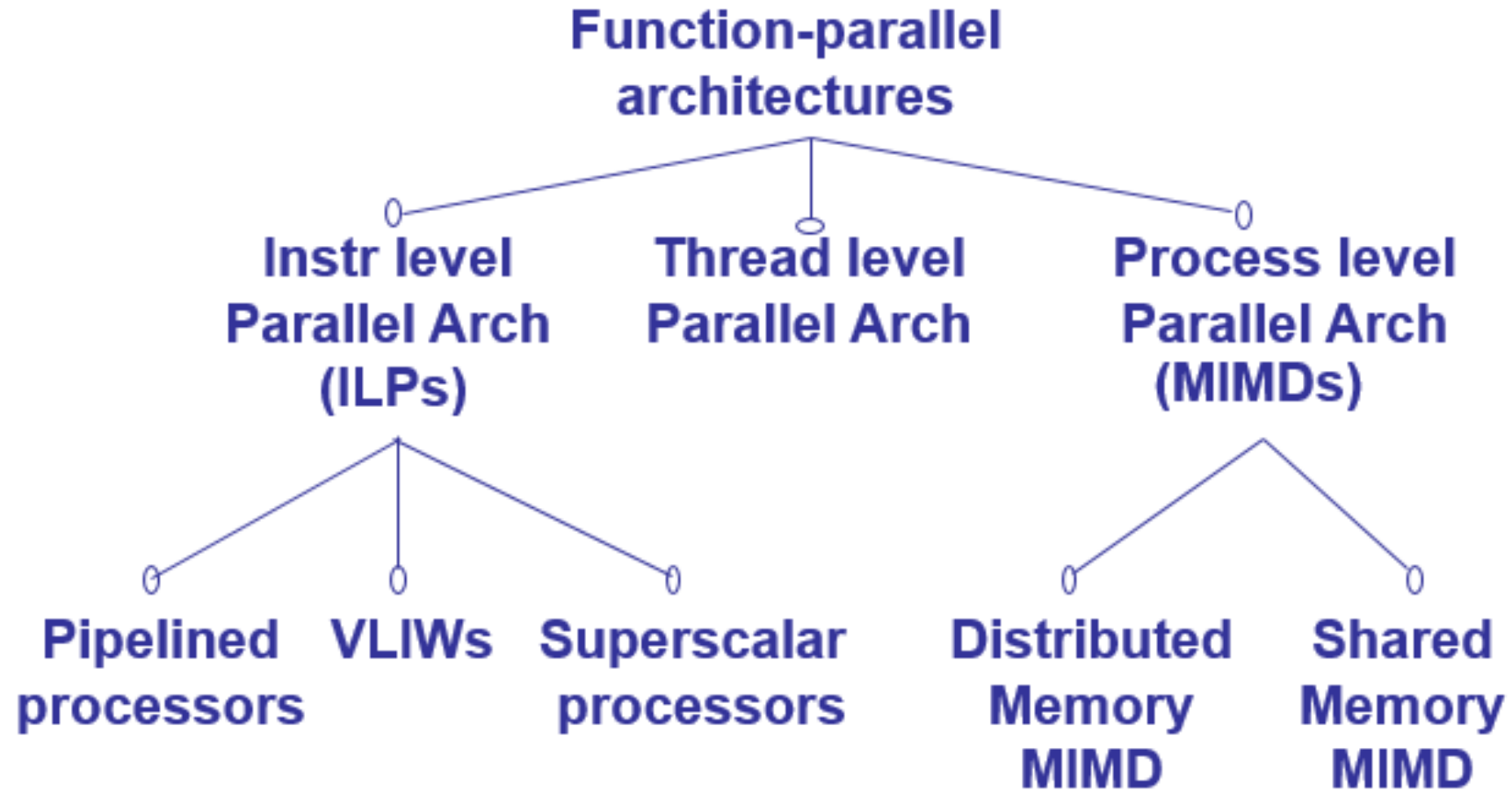
Modern Classification



Data Parallel Architectures



Function Parallel Architectures



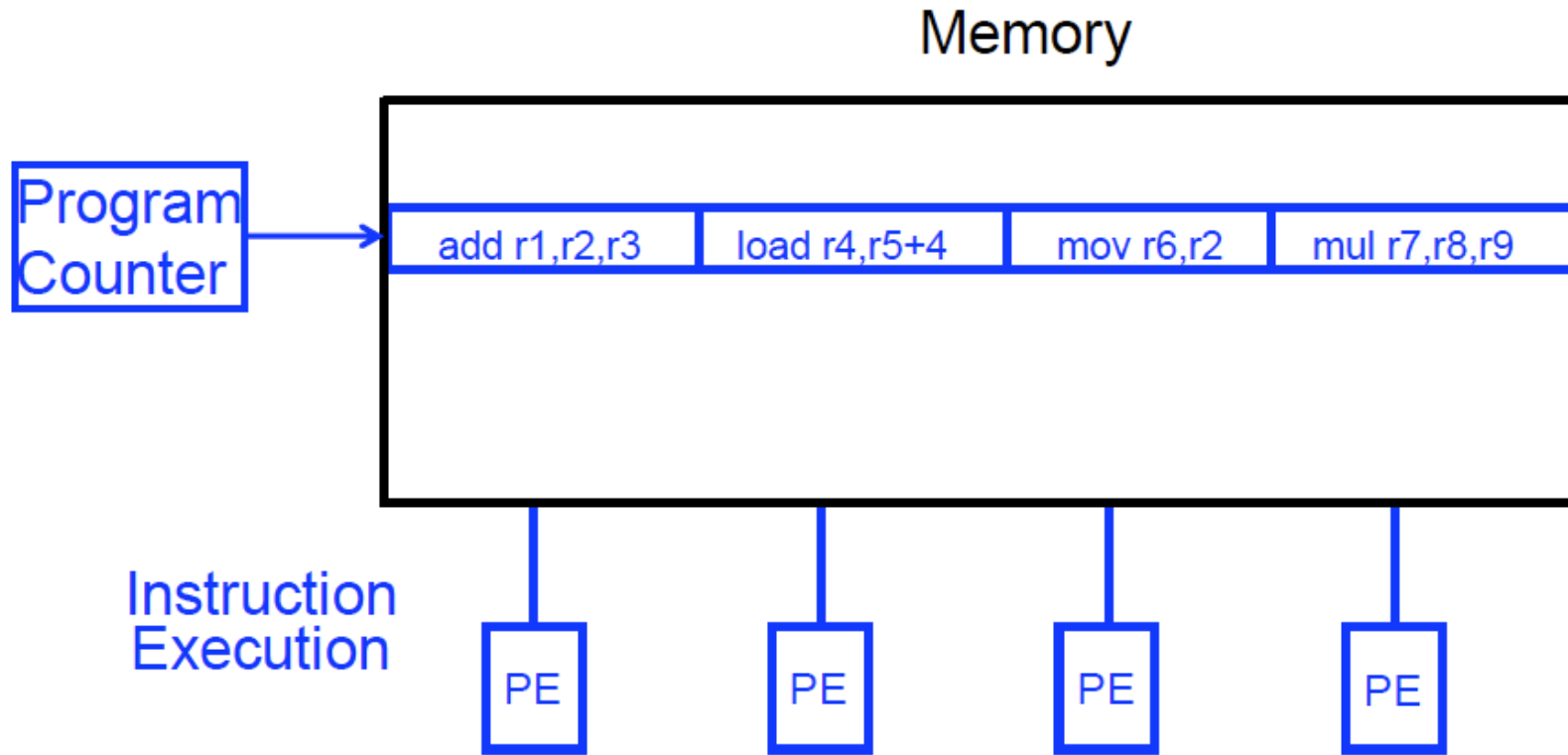
VLIW Architectures

(Very Long Instruction Word)

VLIW Concept

- Superscalar
 - **Hardware** fetches multiple instructions and checks dependencies between them
- VLIW (Very Long Instruction Word)
 - **Software (compiler) packs independent instructions** in a larger “instruction bundle” to be fetched and executed concurrently
 - Hardware fetches and executes the instructions in the bundle concurrently
- **No need for hardware dependency checking** between concurrently-fetched instructions **in the VLIW model**
 - **Simple hardware, complex compiler**

VLIW Concept



- Fisher, “**Very Long Instruction Word architectures and the ELI-512,**” ISCA 1983.
 - ELI: Enormously longword instructions (512 bits)

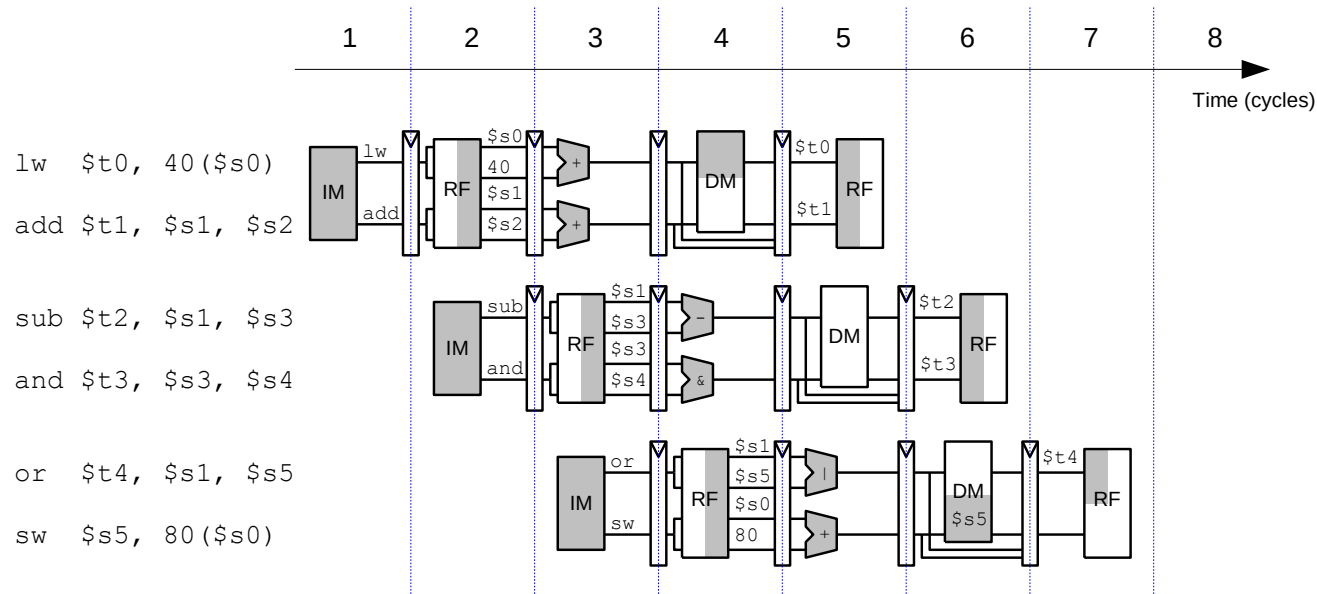
VLIW (Very Long Instruction Word)

- A very long instruction word consists of **multiple independent instructions packed together by the compiler**
 - Packed instructions can be logically unrelated (contrast with SIMD/vector processors, which we will see soon)
- Idea: Compiler finds independent instructions and statically schedules (i.e. packs/bundles) them into a single VLIW instruction
- Traditional VLIW Characteristics
 1. Multiple instruction fetch/execute, multiple functional units
 2. All instructions in a bundle are executed in lock step
 3. Instructions in a bundle statically aligned to be directly supplied into the functional units

VLIW Performance Example (2-wide bundles)

<code>lw \$t0, 40(\$s0)</code>	<code>add \$t1, \$s1, \$s2</code>	Bundle 1
<code>sub \$t2, \$s1, \$s3</code>	<code>and \$t3, \$s3, \$s4</code>	Bundle 2
<code>or \$t4, \$s1, \$s5</code>	<code>sw \$s5, 80(\$s0)</code>	Bundle 3

Ideal IPC = 2



Actual IPC = 2 (6 instructions issued in 3 cycles)

VLIW Lock-Step Execution

- Lock-step (all or none) execution
 - If any operation in a VLIW instruction stalls, all concurrent operations stall
- In a **truly VLIW machine**:
 - the compiler handles all dependency-related stalls
 - hardware does **not** perform dependency checking
 - What about variable latency operations? Memory stalls?

VLIW Philosophy & Principles

Proceedings of the ACM SIGPLAN '84 Symposium on Compiler Construction,
SIGPLAN Notices Vol. 19, No. 6, June 1984

Parallel Processing: A Smart Compiler and a Dumb Machine

Joseph A. Fisher, John R. Ellis,
John C. Ruttenberg, and Alexandru Nicolau

Department of Computer Science, Yale University
New Haven, CT 06520

Abstract

Multiprocessors and vector machines, the only successful parallel architectures, have coarse-grained parallelism that is hard for compilers to take advantage of. We've developed a new fine-grained parallel architecture and a compiler that together offer order-of-magnitude speedups for ordinary scientific code.

future, and we're building a VLIW machine, the ELI (Enormously Long Instructions) to prove it.

In this paper we'll describe some of the compilation techniques used by the Bulldog compiler. The ELI project and the details of Bulldog are described elsewhere [4, 6, 7, 15, 17].

VLIW Philosophy & Principles

- Philosophy similar to RISC (simple instructions and hardware)
 - Except "multiple instructions in parallel: in VLIW"
- RISC (John Cocke+, 1970s, IBM 801 minicomputer)
 - Compiler does the hard work to translate high-level language code to simple instructions (John Cocke: control signals)
 - And, to reorder simple instructions for high performance
 - Hardware does little translation/decoding → very simple
- VLIW (Josh Fisher, ISCA 1983)
 - Compiler does the hard work to find instruction level parallelism
 - Hardware stays as simple as possible
 - Executes each instruction in a bundle in lock step
 - Simple → higher frequency, easier to design, low power

VLIW Philosophy and Properties

More formally, VLIW architectures have the following properties:

There is one central control unit issuing a single long instruction per cycle.

Each long instruction consists of many tightly coupled independent operations.

Each operation requires a small, statically predictable number of cycles to execute.

Operations can be pipelined. These properties distinguish VLIWs from multiprocessors (with large asynchronous tasks) and dataflow machines (without a single flow of control, and without the tight coupling). VLIWs have none of the required regularity of a vector processor, or true array processor.

Commercial VLIW Machines

- Multiflow TRACE, Josh Fisher (7-wide, 28-wide)
- Cydrome Cydra 5, Bob Rau
- Transmeta Crusoe: x86 binary-translated into internal VLIW
- TI C6000, Trimedia, STMicro (DSP & embedded processors) and some ATI/AMD GPUs
 - ❑ Most successful commercially
- Intel IA-64
 - ❑ Not fully VLIW, but based on VLIW principles
 - ❑ EPIC (Explicitly Parallel Instruction Computing)
 - ❑ Instruction bundles can have dependent instructions
 - ❑ A few bits in the instruction format specify explicitly which instructions in the bundle are dependent on which other ones

VLIW Tradeoffs

■ Advantages

- + No need for dynamic scheduling hardware → **simple hardware**
- + No need for dependency checking within a VLIW instruction → **simple hardware** for multiple instruction issue + no renaming
- + No need for instruction alignment/distribution after fetch to different functional units → **simple hardware**

■ Disadvantages

- **Compiler** needs to find N independent operations per cycle
 - If it cannot, inserts **NOPs** in a VLIW instruction
 - Parallelism loss AND code size increase
- **Recompilation required** when execution width (N), instruction latencies, functional units change (Unlike superscalar processing)
- **Lockstep execution** causes independent operations to stall
 - No instruction can progress until the longest-latency instruction completes

VLIW Summary

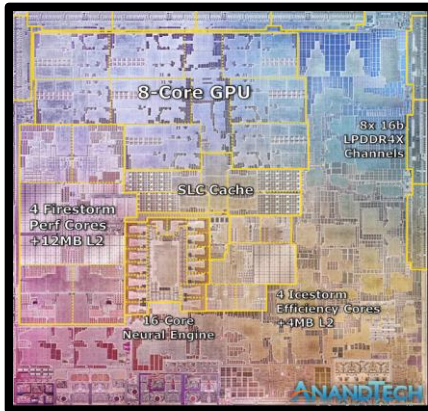
- VLIW simplifies hardware, but requires complex compiler techniques
 - Solely-compiler approach of VLIW has several downsides that reduce performance
 - No tolerance for variable or long-latency operations (lock step)
 - Too many NOPs (not enough parallelism discovered)
 - Static schedule intimately tied to microarchitecture
 - Code optimized for one generation performs poorly for next
- ++ Most compiler optimizations developed for VLIW employed in optimizing compilers (for superscalar compilation)
- Enable code optimizations
- ++ VLIW very successful when parallelism is easier to find by the compiler (traditionally embedded markets, DSPs, GPUs)

Systolic Arrays

General Purpose vs. Special Purpose Systems

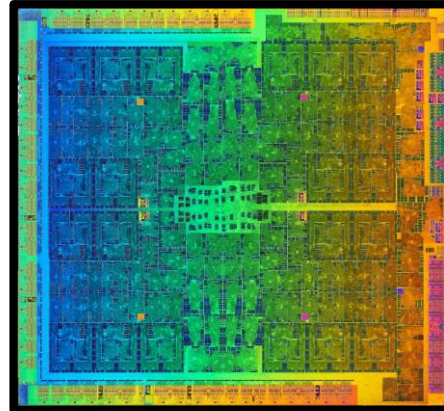
General Purpose

CPU_s



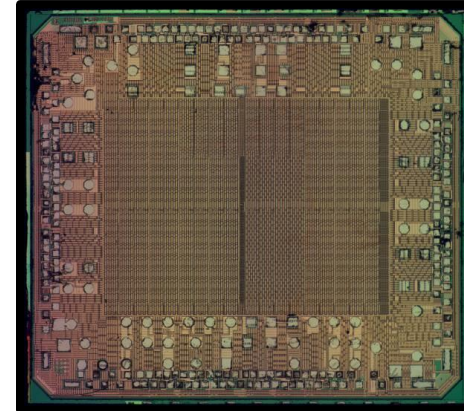
Apple M1

GPU_s



Nvidia GTX 1070

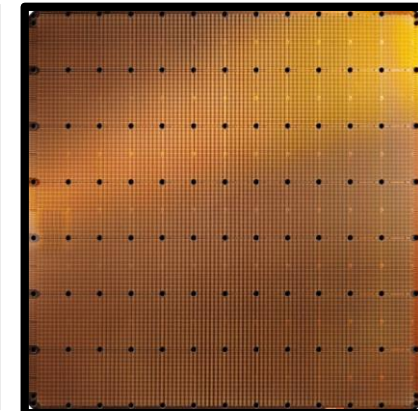
FPGAs



Xilinx Spartan

Special Purpose

ASIC_s



Cerebras WSE-2



Flexible: Can execute any program
Easy to program & use
Not the best performance & efficiency

Efficient & High performance
(Usually) Difficult to program & use
Inflexible: Limited set of programs

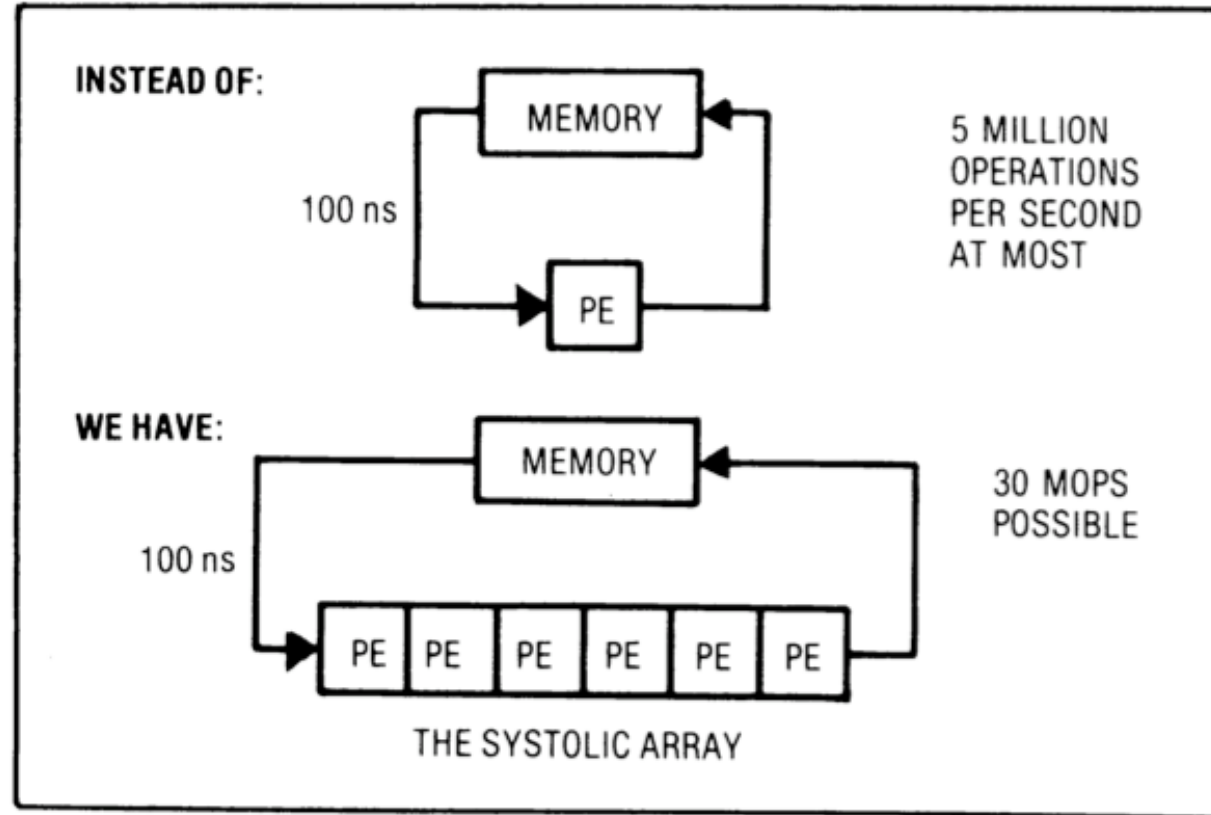
Systolic Arrays: Specialized Accelerators

- **Systolic arrays comprise an execution model**
 - They implement systolic computation
 - Different from **von Neumann**, different from **dataflow**
- They are initially designed as **special-purpose accelerators**
 - For convolutions, filtering, pattern matching, special-purpose matrix/vector computations in image & vision processing, signal processing, pattern recognition, etc.
- They are currently used heavily in machine learning
 - **Specialized machine learning accelerators**
- Their general execution model can be (and is) generalized
 - As we will see later in this lecture

Systolic Arrays: Motivation

- Goal: design an accelerator that has
 - Simple, regular design (keep # unique parts small and regular)
 - High concurrency → high performance
 - Balanced computation and I/O (memory) bandwidth
- Idea: Replace a single processing element (PE) with a regular array of PEs and carefully orchestrate flow of data between the PEs
 - such that they collectively transform a piece of input data before outputting it to memory
- Benefit: Maximizes computation done on a single piece of data element brought from memory

Systolic Arrays



Memory: heart
Data: blood
PEs: cells

Memory pulses
data through
PEs

Figure 1. Basic principle of a systolic system.

- H. T. Kung, "[Why Systolic Architectures?](#)," IEEE Computer 1982.

Why Systolic Architectures?

- Idea: Data flows from the computer memory in a rhythmic fashion, passing through many processing elements before it returns to memory
- Similar to blood flow: heart → many cells → heart
 - Different cells “process” the blood
 - Many veins operate simultaneously
 - Can be many-dimensional
- Why? Special purpose accelerators/architectures need
 - Simple, regular design (keep # unique parts small and regular)
 - High concurrency → high performance
 - Balanced computation and I/O (memory) bandwidth

Systolic Architectures

- Basic principle: Replace a single PE with a **regular array of PEs** and **carefully orchestrate flow of data** between the PEs
 - Balance computation and memory bandwidth

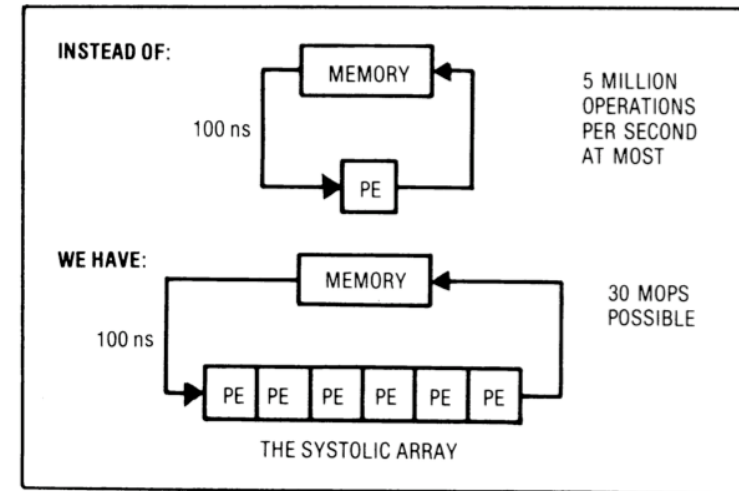


Figure 1. Basic principle of a systolic system.

- Differences from pipelining:
 - These are individual PEs
 - Array structure can be non-linear and multi-dimensional
 - PE connections can be multidirectional (and different speed)
 - PEs can have local memory and execute kernels (rather than a piece of the instruction)

Systolic Computation Example

■ Convolution

- Used in filtering, pattern matching, correlation, polynomial evaluation, etc ...
- Many **image processing** tasks
- **Machine learning**: up to hundreds of **convolutional layers** in Convolutional Neural Networks (CNN)

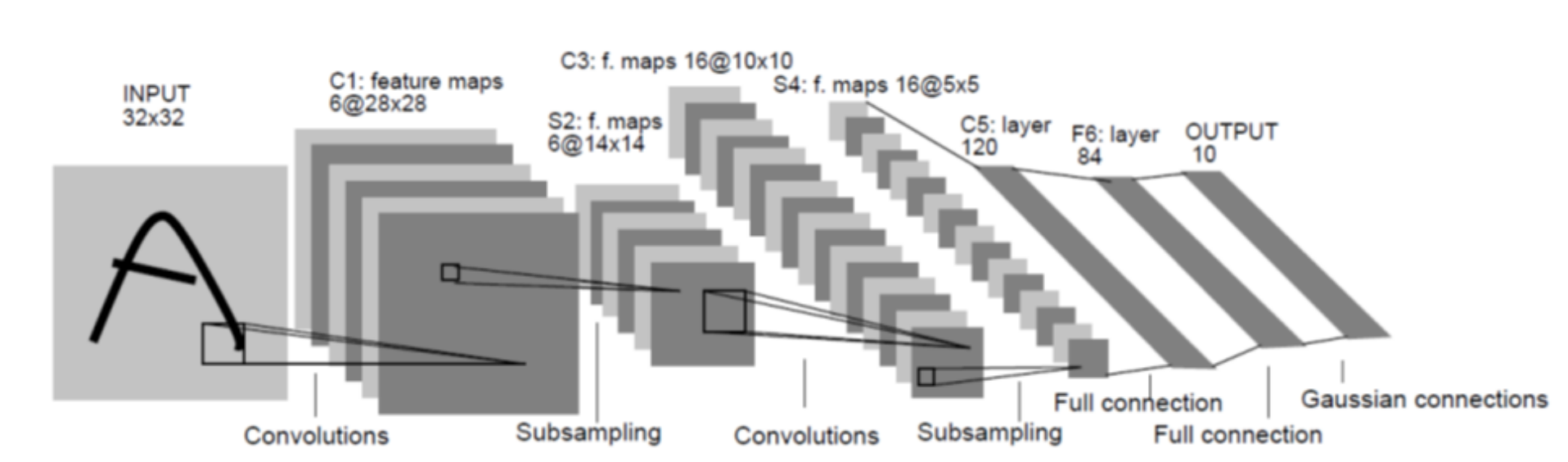
Given the sequence of weights $\{w_1, w_2, \dots, w_k\}$
and the input sequence $\{x_1, x_2, \dots, x_n\}$,

compute the result sequence $\{y_1, y_2, \dots, y_{n+1-k}\}$
defined by

$$y_i = w_1x_i + w_2x_{i+1} + \dots + w_kx_{i+k-1}$$

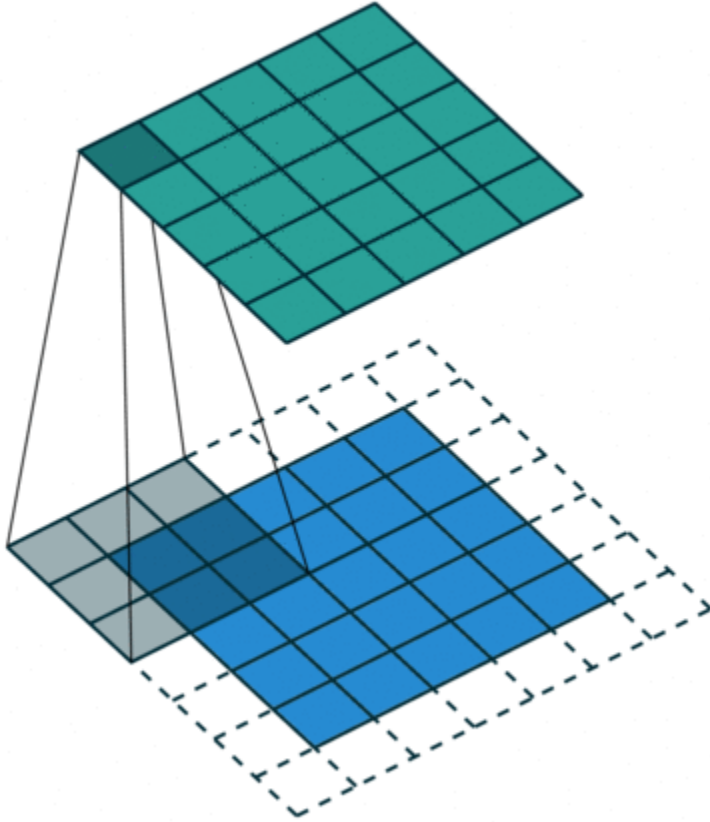
LeNet-5, a Convolutional Neural Network for Hand-Written Digit Recognition

This is a 1024*8 bit input, which will have a truth table of 2^{8196} entries



An Example of 2D Convolution

Output feature map



Input feature map

Structure information

Input: 5*5 (blue)

Kernel (filter): 3*3 (grey)

Output: 5*5 (green)

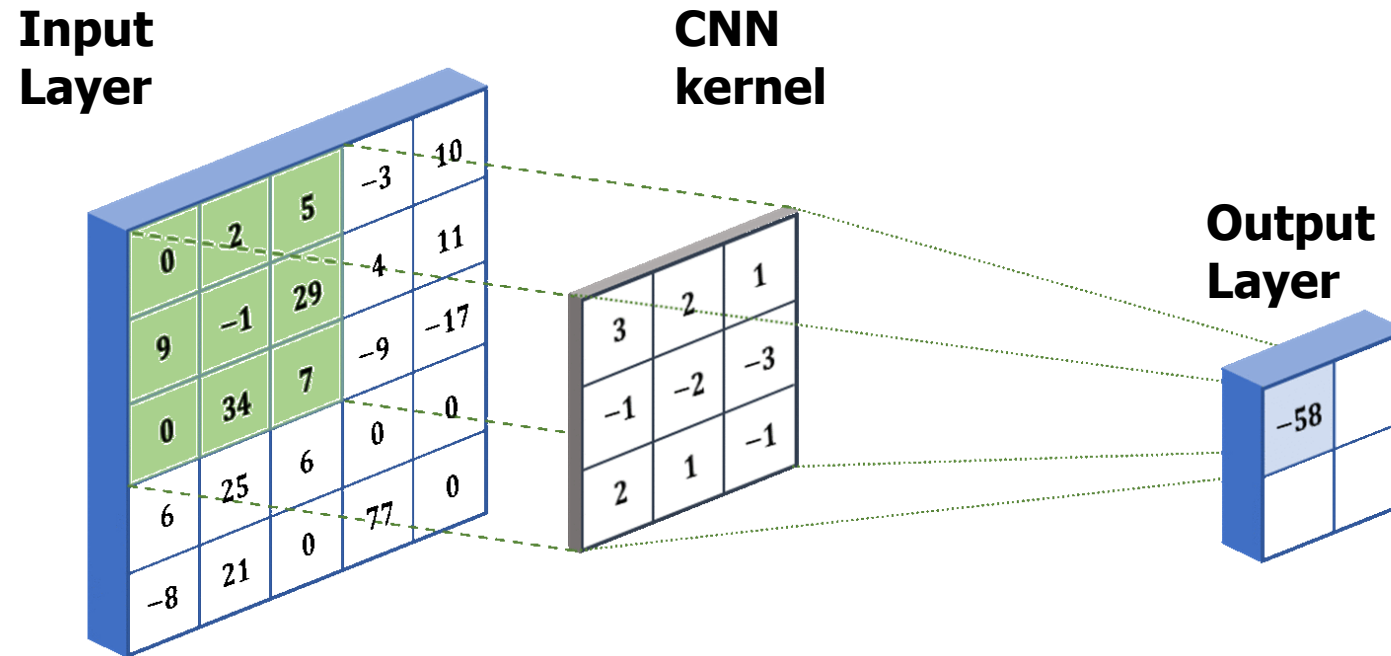
Computation information

Stride: 1

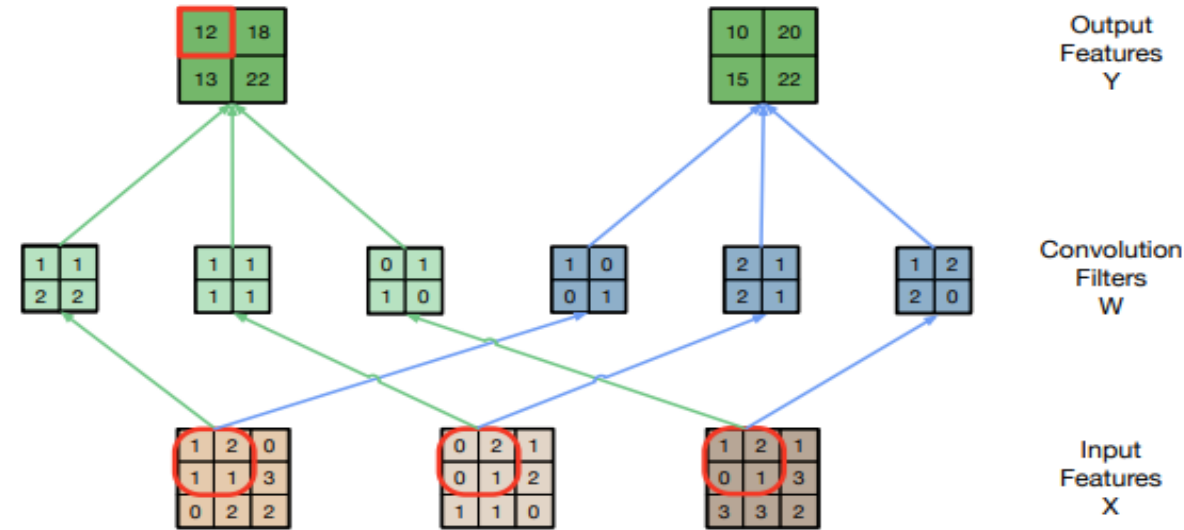
Padding: 1 (white)

$$\text{Output Dim} = (\text{Input} + 2 * \text{Padding} - \text{Kernel}) / \text{Stride} + 1$$

An Example of 2D Convolution



Implementing a Convolutional Layer with Matrix Multiplication



$$\begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 2 & 2 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ \hline 1 & 0 & 0 & 1 & 2 & 1 & 2 & 1 & 1 & 2 & 2 & 0 \\ \hline \end{array} * \begin{array}{|c|c|c|c|} \hline 1 & 2 & 1 & 1 \\ \hline 2 & 0 & 1 & 3 \\ \hline 1 & 1 & 0 & 2 \\ \hline 1 & 3 & 2 & 2 \\ \hline 0 & 2 & 0 & 1 \\ \hline 2 & 1 & 1 & 2 \\ \hline 0 & 1 & 1 & 1 \\ \hline 1 & 2 & 1 & 0 \\ \hline 1 & 2 & 0 & 1 \\ \hline 2 & 1 & 1 & 3 \\ \hline 0 & 1 & 3 & 3 \\ \hline 1 & 3 & 3 & 2 \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 12 & 18 & 13 & 22 \\ \hline 10 & 20 & 15 & 22 \\ \hline \end{array}$$

Convolution Filters W' Input Features X (unrolled) Output Features Y

Systolic Computation Example: Convolution (I)

■ Convolution

- Used in filtering, pattern matching, correlation, polynomial evaluation, etc ...
- Many **image processing** tasks
- **Machine learning**: up to hundreds of **convolutional layers** in Convolutional Neural Networks (CNN)

Given the sequence of weights $\{w_1, w_2, \dots, w_k\}$
and the input sequence $\{x_1, x_2, \dots, x_n\}$,

compute the result sequence $\{y_1, y_2, \dots, y_{n+1-k}\}$
defined by

$$y_i = w_1x_i + w_2x_{i+1} + \dots + w_kx_{i+k-1}$$

Systolic Computation Example: Convolution (I)

- $y_1 = w_1x_1 + w_2x_2 + w_3x_3$
- $y_2 = w_1x_2 + w_2x_3 + w_3x_4$
- $y_3 = w_1x_3 + w_2x_4 + w_3x_5$

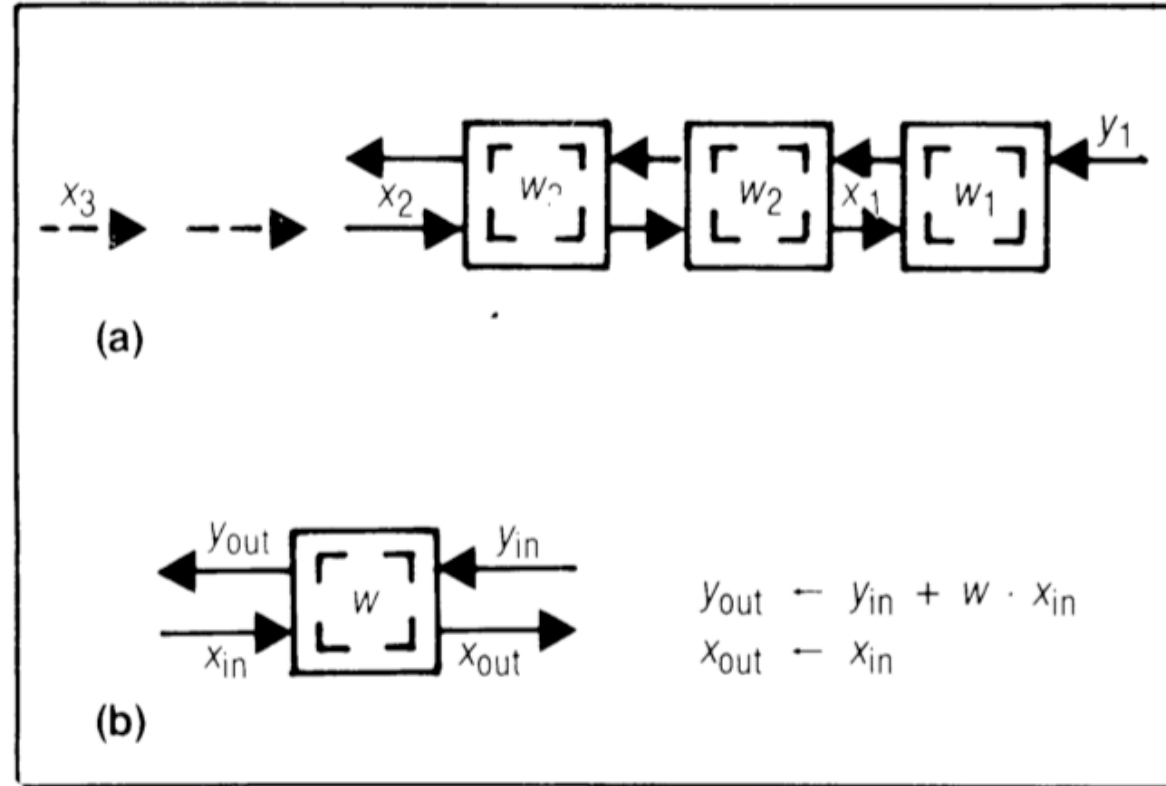


Figure 8. Design W1: systolic convolution array (a) and cell (b) where w_i 's stay and x_i 's and y_i 's move systolically in opposite directions.

Systolic Computation Example: Convolution (II)

- One needs to **carefully orchestrate** when **data elements** are **input to the array**
- And when **output is buffered**
- This gets more involved when
 - Array dimensionality increases
 - PEs are less predictable in terms of latency

Example 2D Systolic Array Computation

- Multiply two 3x3 matrices (inputs)
 - Keep the final result in PE accumulators

$$\begin{bmatrix} c_{00} & c_{01} & c_{02} \\ c_{10} & c_{11} & c_{12} \\ c_{20} & c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ a_{20} & a_{21} & a_{22} \end{bmatrix} \times \begin{bmatrix} b_{00} & b_{01} & b_{02} \\ b_{10} & b_{11} & b_{12} \\ b_{20} & b_{21} & b_{22} \end{bmatrix}$$

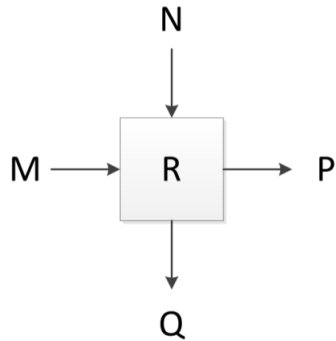
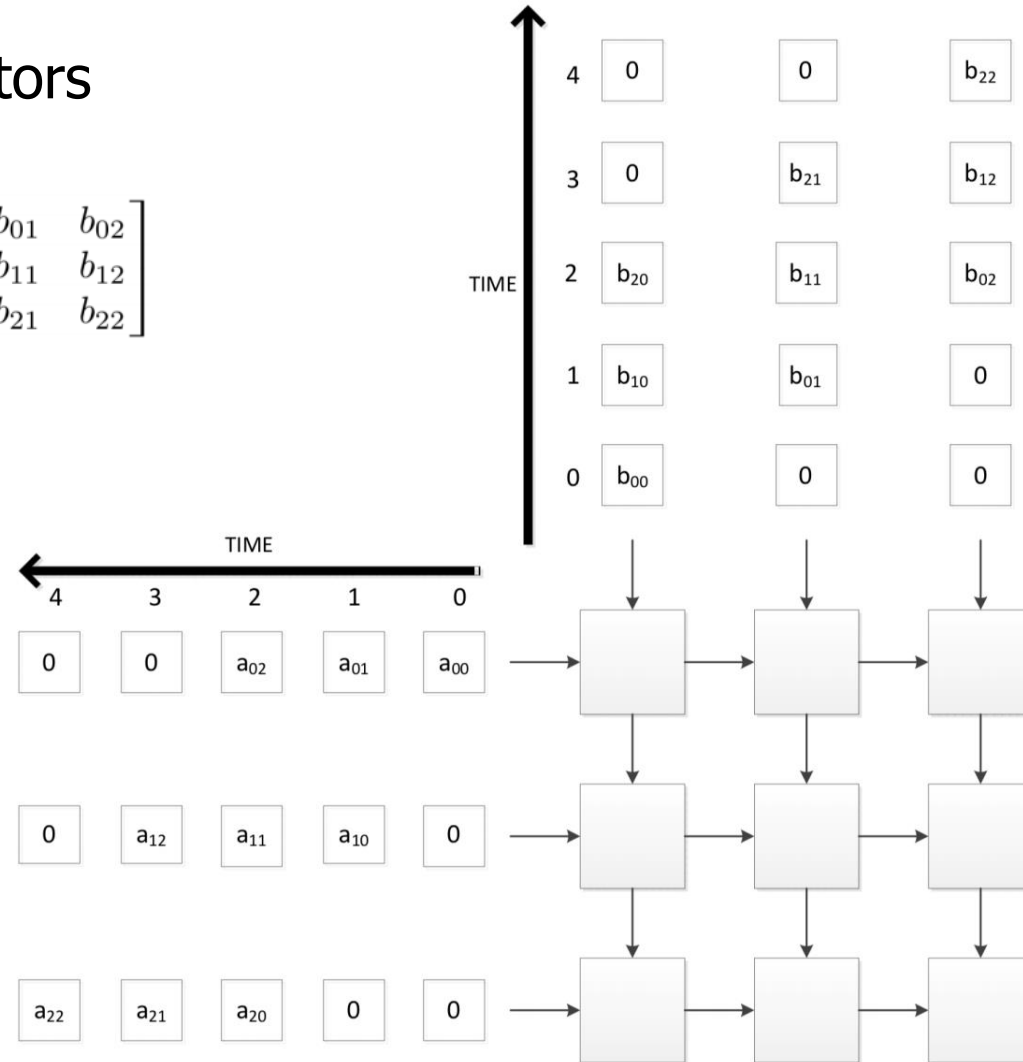


Figure 1: A systolic array processing element

$$P = M$$

$$Q = N$$

$$R = R + M * N$$



Systolic Arrays: Pros and Cons

- Advantages:

- **Principled**: Efficiently makes use of limited memory bandwidth, balances computation to I/O bandwidth availability
- **Specialized** (computation should fit the PE organization/functions)
 - improved **efficiency, simple design, high concurrency/performance**
 - good to do **more with less memory bandwidth** requirement

- Downside:

- **Specialized**
 - **not generally applicable** because computation needs to fit the PE functions/organization

More Programmability in Systolic Arrays

- Each PE in a systolic array
 - Can store multiple “weights”
 - Weights can be selected on the fly
 - Eases implementation of, e.g., adaptive filtering
- Taken further
 - Each PE can have its own data and instruction memory
 - Data memory → to store partial/temporary results, constants
 - Leads to **stream processing, pipeline parallelism**
 - More generally, **staged execution**

Systolic Array: Advantages & Disadvantages

■ Advantages

- ❑ Special purpose → high efficiency
- ❑ Makes multiple uses of each data item → reduced need for fetching/refetching → better use of memory bandwidth
- ❑ High concurrency
- ❑ Regular design (both data and control flow) → easier to implement

■ Disadvantages

- ❑ Not good at exploiting irregular parallelism
- ❑ Special purpose → not generally applicable
 - Needs software, programmer support to become more general purpose
- ❑ Difficult to program if problem does not fit (well)

An Example Modern Systolic Array: TPU (I)

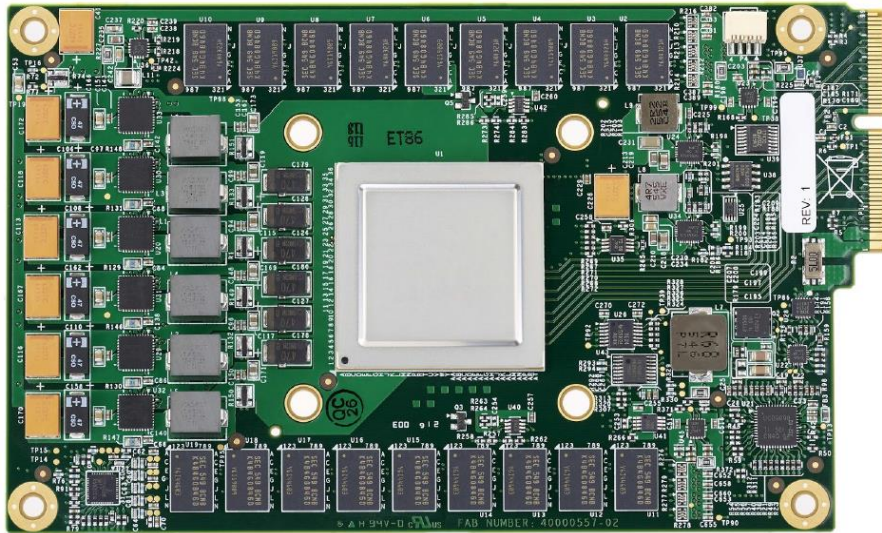


Figure 3. TPU Printed Circuit Board. It can be inserted in the slot for an SATA disk in a server, but the card uses PCIe Gen3 x16.

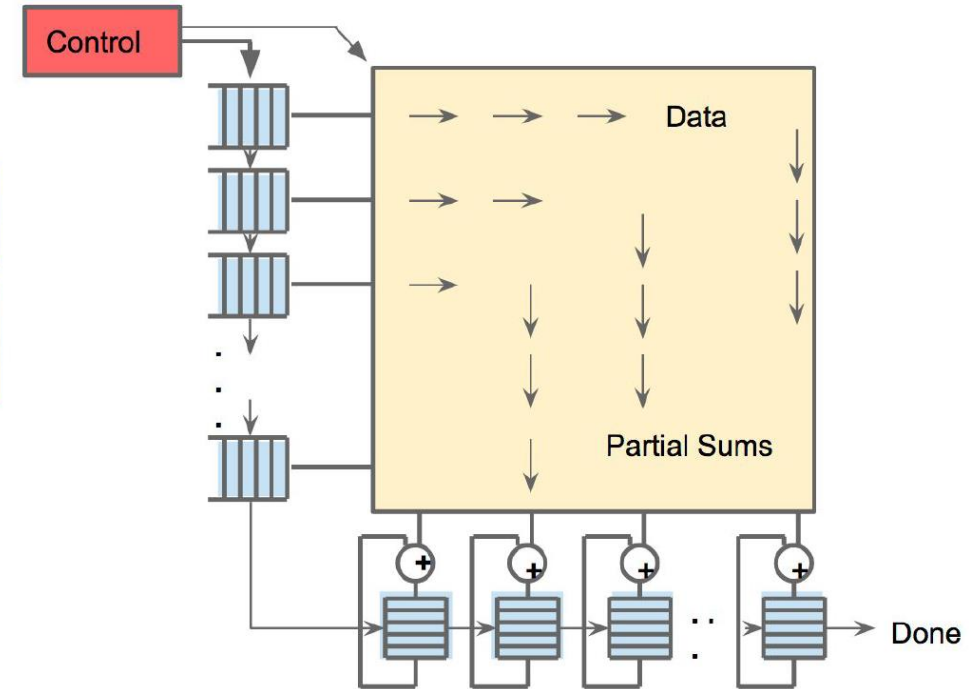
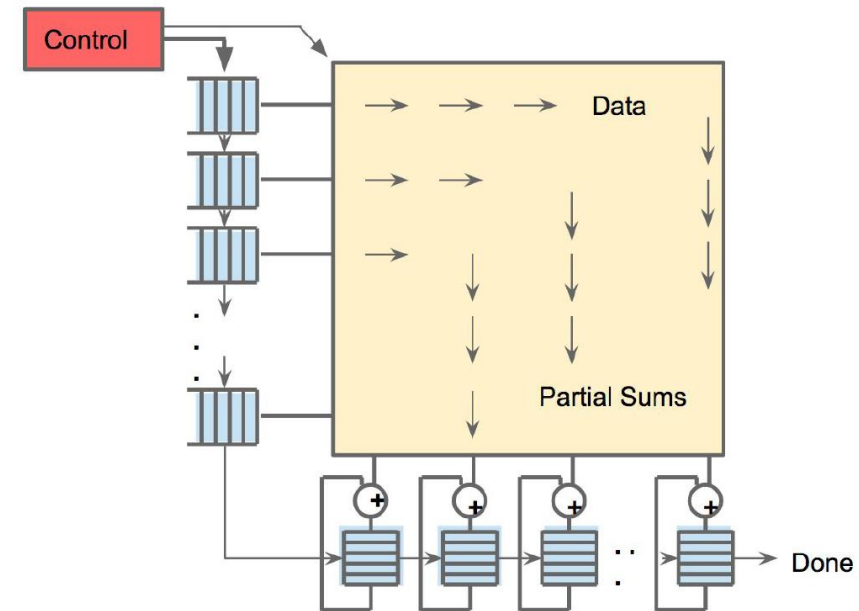


Figure 4. Systolic data flow of the Matrix Multiply Unit. Software has the illusion that each 256B input is read at once, and they instantly update one location of each of 256 accumulator RAMs.

Jouppi et al., “In-Datacenter Performance Analysis of a Tensor Processing Unit”, ISCA 2017.

An Example Modern Systolic Array: TPU (II)

As reading a large SRAM uses much more power than arithmetic, the matrix unit uses systolic execution to save energy by reducing reads and writes of the Unified Buffer [Kun80][Ram91][Ovt15b]. Figure 4 shows that data flows in from the left, and the weights are loaded from the top. A given 256-element multiply-accumulate operation moves through the matrix as a diagonal wavefront. The weights are preloaded, and take effect with the advancing wave alongside the first data of a new block. Control and data are pipelined to give the illusion that the 256 inputs are read at once, and that they instantly update one location of each of 256 accumulators. From a correctness perspective, software is unaware of the systolic nature of the matrix unit, but for performance, it does worry about the latency of the unit.



Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit", ISCA 2017.

An Example Modern Systolic Array: TPU2



<https://www.nextplatform.com/2017/05/17/first-depth-look-googles-new-second-generation-tpu/>

4 TPU chips
vs 1 chip in TPU1

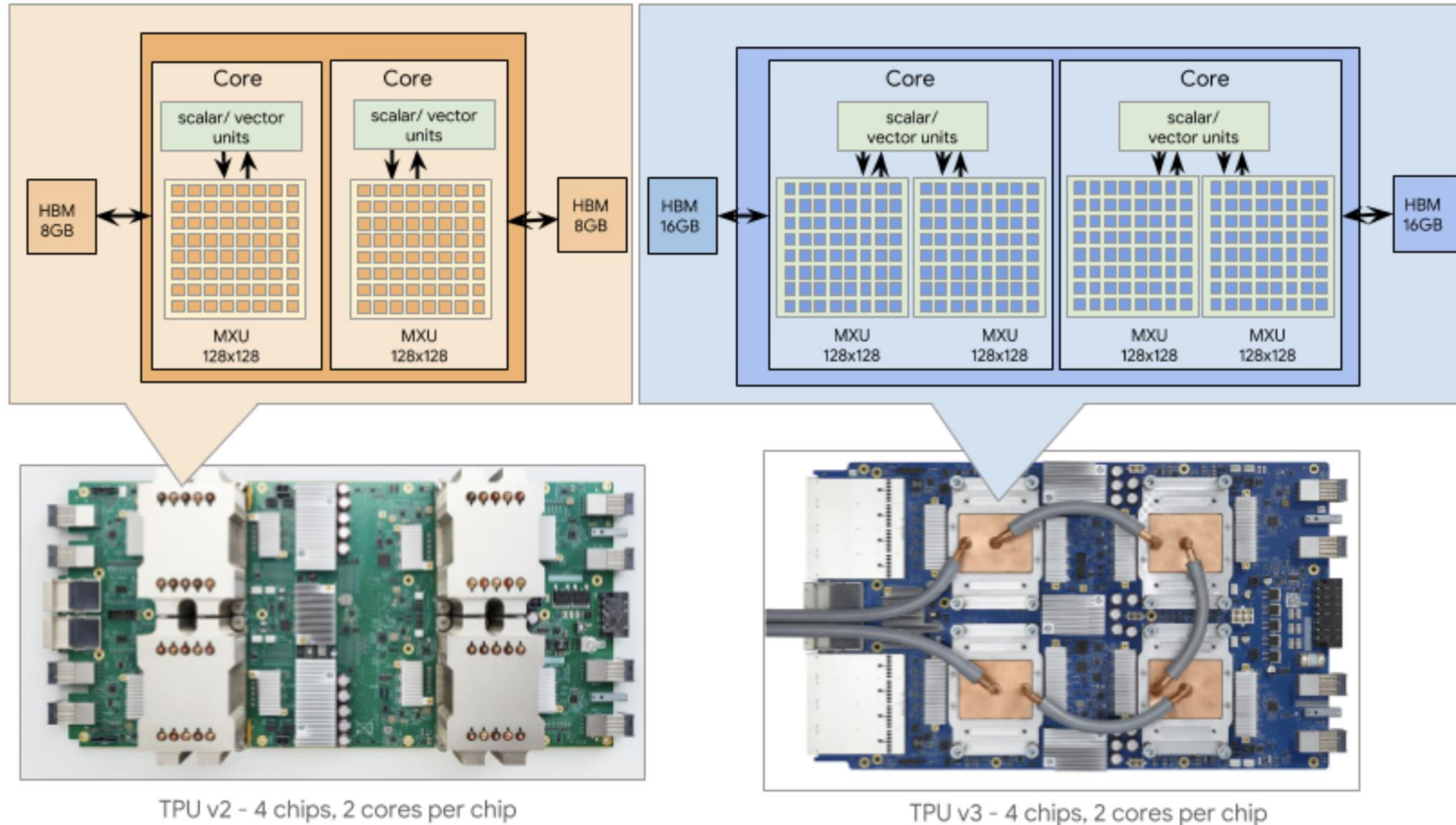
High Bandwidth Memory
vs DDR3

Floating point operations
vs FP16

45 TFLOPS per chip
vs 23 TOPS

Designed for training
and inference
vs only inference

An Example Modern Systolic Array: TPU3

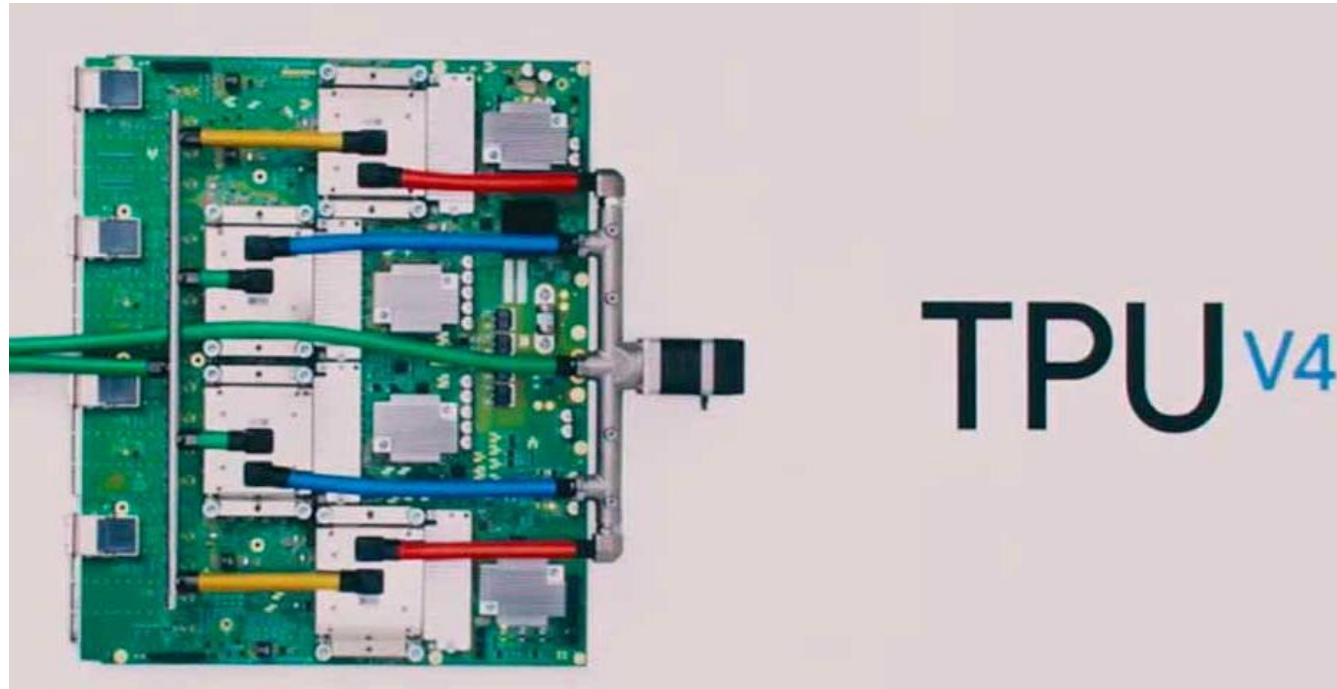


32GB HBM per chip
vs 16GB HBM in TPU2

4 Matrix Units per chip
vs 2 Matrix Units in TPU2

90 TFLOPS per chip
vs 45 TFLOPS in TPU2

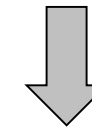
An Example Modern Systolic Array: TPU4



New ML applications (vs. TPU3):

- Computer vision
- Natural Language Processing (NLP)
- Recommender system
- Reinforcement learning that plays Go

250 TFLOPS per chip in 2021
vs 90 TFLOPS in TPU3



1 ExaFLOPS per board

<https://spectrum.ieee.org/tech-talk/computing/hardware/heres-how-googles-tpu-v4-ai-chip-stacked-up-in-training-tests>

A Summary Reading of Google TPUs

Ten Lessons From Three Generations Shaped Google's TPUv4i

Industrial Product

Norman P. Jouppi, Doe Hyun Yoon, Matthew Ashcraft, Mark Gottscho, Thomas B. Jablin, George Kurian, James Laudon, Sheng Li, Peter Ma, Xiaoyu Ma, Thomas Norrie, Nishant Patil, Sushma Prasad, Cliff Young, Zongwei Zhou, and David Patterson, Google LLC

A Recent Reading on the TPUv4

TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings Industrial Product*

Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson
Google, Mountain View, CA

Exploiting Data Parallelism: SIMD Architectures and GPUs

Flynn's Taxonomy of Computers

- Mike Flynn, “[Very High-Speed Computing Systems](#),” Proc. of IEEE, 1966
- **SISD**: Single instruction operates on single data element
- **SIMD**: Single instruction operates on multiple data elements
 - Array processor
 - Vector processor
- **MISD**: Multiple instructions operate on single data element
 - Closest form: systolic array processor, streaming processor
- **MIMD**: Multiple instructions operate on multiple data elements (multiple instruction streams)
 - Multiprocessor
 - Multithreaded processor

Data Parallelism

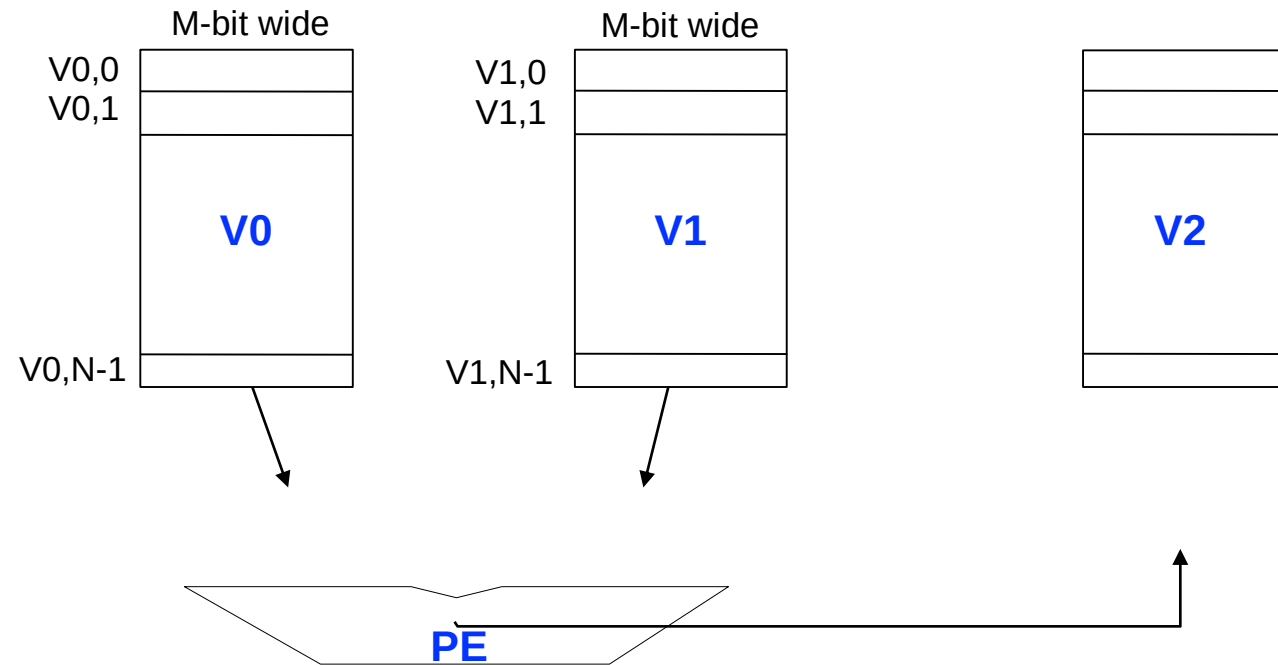
- Concurrency arises from performing the **same operation on different pieces of data**
 - Single instruction multiple data (SIMD)
 - E.g., dot product of two vectors
- Contrast with data flow
 - Concurrency arises from executing different operations in parallel (in a data driven manner)
- Contrast with thread (“control”) parallelism
 - Concurrency arises from executing different threads of control in parallel
- SIMD exploits operation-level parallelism on different data
 - Same operation concurrently applied to different pieces of data
 - A form of ILP where instruction happens to be the same across data

SIMD Processing Paradigm

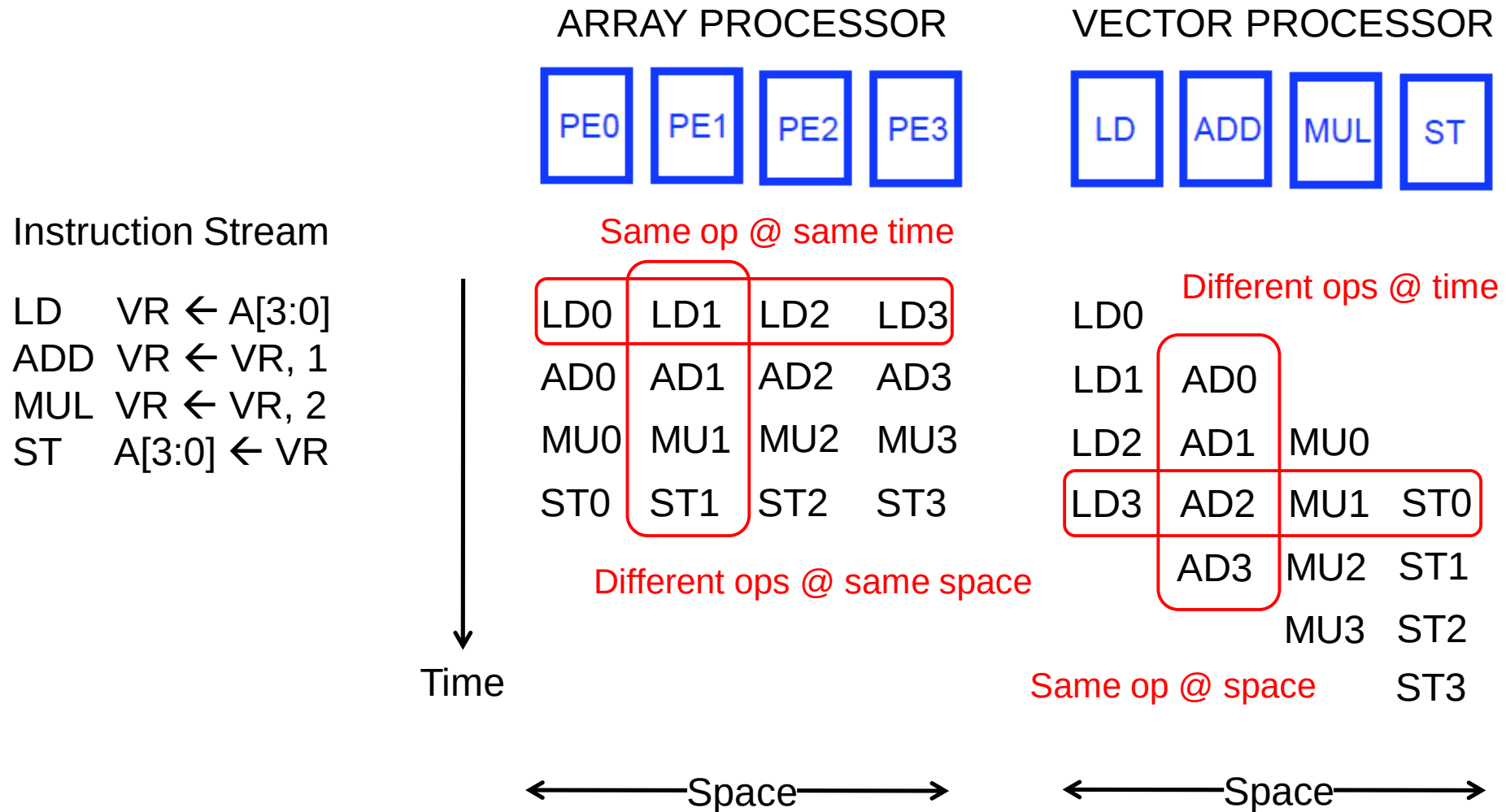
- Single instruction operates on multiple data elements
 - In time or in space
- Multiple processing elements (PEs), i.e., execution units
- Time-space duality
 - **Array processor**: Instruction operates on multiple data elements at the **same time** using **different spaces (PEs)**
 - **Vector processor**: Instruction operates on multiple data elements in **consecutive time steps** using the **same space (PE)**

Storing Multiple Data Elements: Vector Registers

- Each **vector data register** holds N M-bit values
 - Each register stores a vector
 - Not a (single) scalar value as we saw before

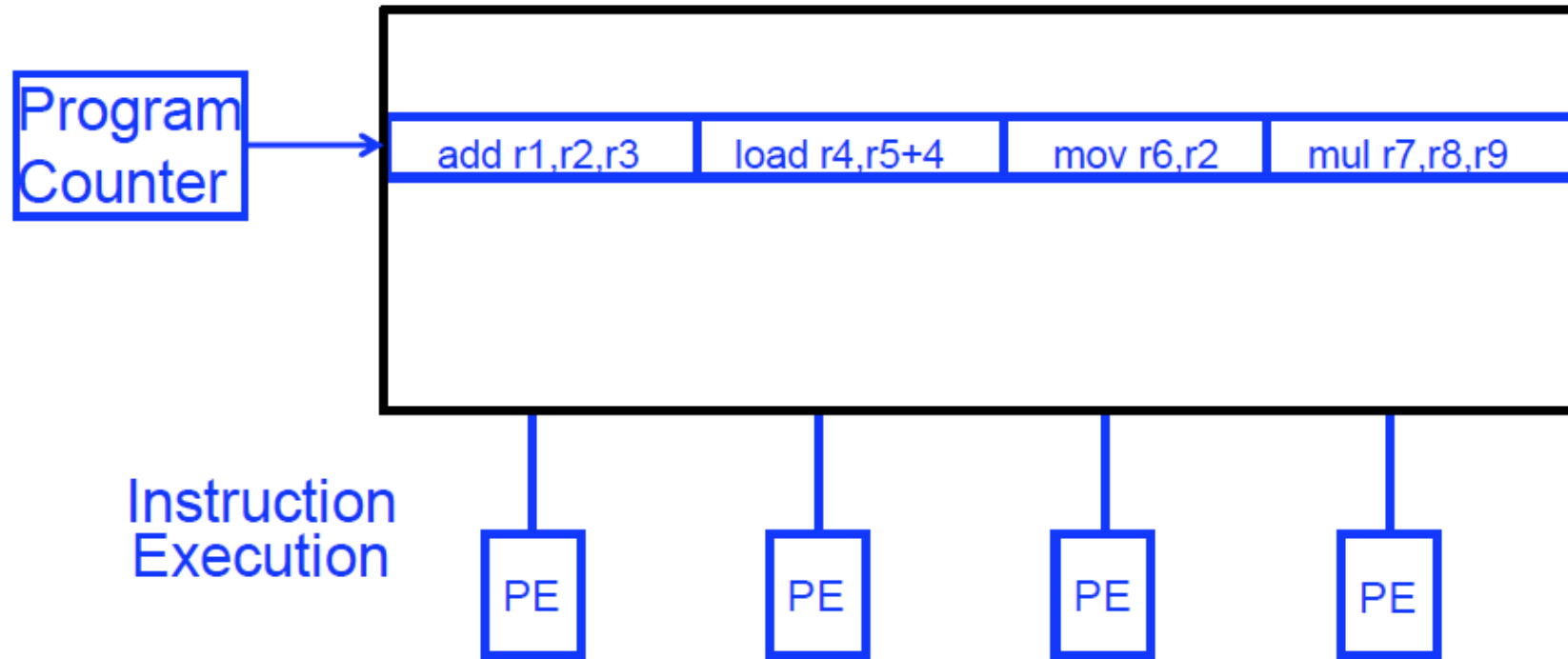


Array vs. Vector Processors



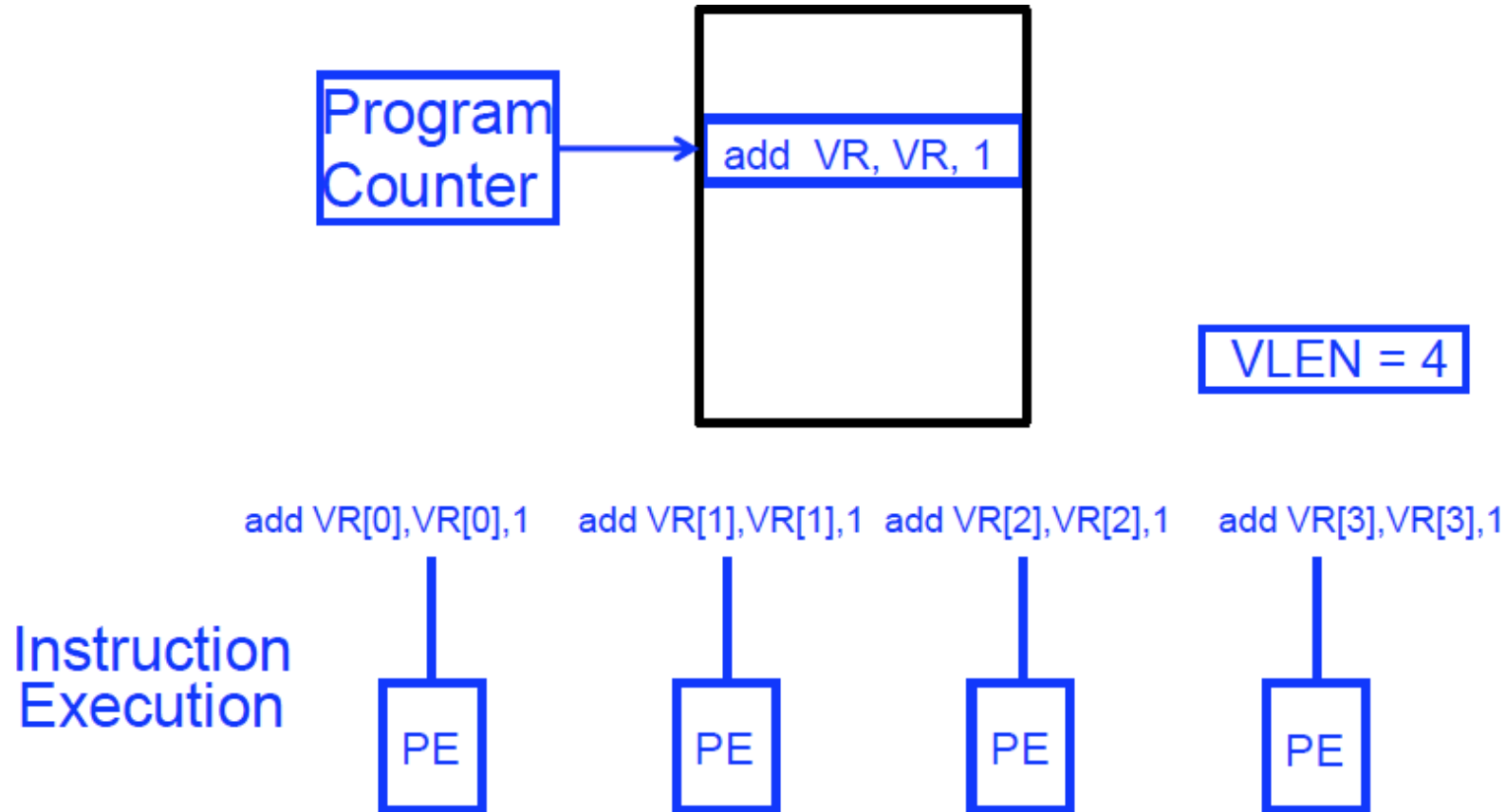
SIMD Array Processing vs. VLIW

- VLIW: **Multiple** independent **operations** packed together into a “long inst.”



SIMD Array Processing vs. VLIW

- Array processor: **Single operation** on multiple (different) data elements



Vector Processors (I)

- A vector is a one-dimensional array of numbers
- Many scientific/commercial programs use vectors
 - for (i = 0; i<=49; i++)
C[i] = (A[i] + B[i]) / 2
- A vector processor is one whose instructions operate on vectors rather than scalar (single data) values
- Basic requirements
 - Need to load/store vectors → vector registers (contain vectors)
 - Need to operate on vectors of different lengths → vector length register (VLEN)
 - Elements of a vector might be stored apart from each other in memory → vector stride register (VSTR)
 - Stride: distance in memory between two elements of a vector

Vector Stride Example: Matrix Multiply

- A and B matrices, both stored in memory in **row-major order**

A_0

0	1	2	3	4	5
6	7	8	9	10	11

$$A_{4 \times 6} B_{6 \times 10} \rightarrow C_{4 \times 10}$$

Dot product of each row vector of A with each column vector of B

B_0

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17	18	19
20									
30									
40									
50									

Linear Memory

A

0
1
2
3
4
5
6

B

0
1
2
3
4
5
6
7
8
9
10

- Load A's row 0 (A0 through A5) into vector register V_1
 - Each time, increment address by **1** to access the next column
 - Accesses have a **stride of 1**
- Load B's column 0 (B0 through B50) into vector register V_2
 - Each time, increment address by **10** to access the next row
 - Accesses have a **stride of 10**

Vector Processors (II)

- A vector instruction performs an operation on each element in consecutive cycles
 - Vector functional units are pipelined
 - Each pipeline stage operates on a different data element
- Vector instructions allow deeper pipelines
 - No intra-vector dependencies → no hardware interlocking needed within a vector
 - No control flow within a vector
 - Known stride allows easy address calculation for all vector elements
 - Enables easy loading (or even early loading, i.e., prefetching) of vectors into registers/cache/memory

Vector Processor Advantages

+ No dependencies within a vector

- ❑ Pipelining & parallelization work really well
- ❑ Can have very deep pipelines (without the penalty of deep pipelines)

+ Each instruction generates a lot of work (i.e., operations)

- ❑ Reduces instruction fetch bandwidth requirements
- ❑ Amortizes instruction fetch and control overhead over many data
 - > Leads to high energy efficiency per operation

+ No need to explicitly code loops

- ❑ Fewer branches in the instruction sequence

+ Highly regular memory access pattern

Vector Processor Disadvantages

- Works (only) if parallelism is regular (data/SIMD parallelism)
 - ++ Vector operations
 - Very inefficient if parallelism is irregular
 - How about searching for a key in a linked list?

To program a vector machine, the compiler or hand coder must make the data structures in the code fit nearly exactly the regular structure built into the hardware. That's hard to do in first place, and just as hard to change. One tweak, and the low-level code has to be rewritten by a very smart and dedicated programmer who knows the hardware and often the subtleties of the application area. Often the rewriting is

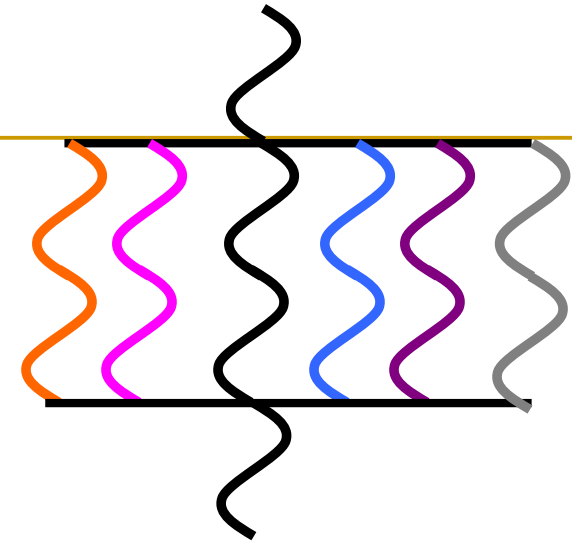
Amdahl's Law

- Amdahl's Law
 - f: Parallelizable fraction of a program
 - N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

- Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” AFIPS 1967.

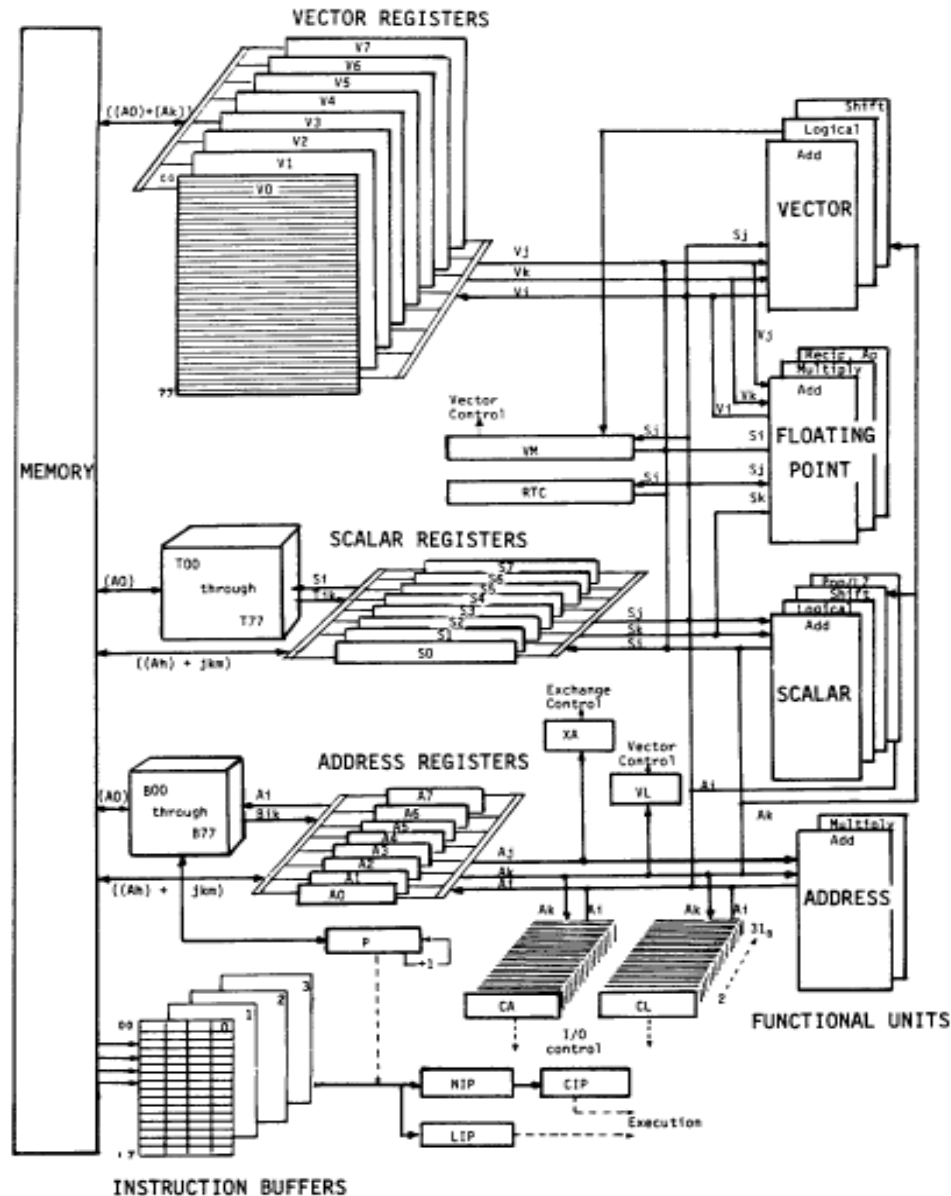
- Maximum speedup limited by serial portion: Serial bottleneck
- All parallel machines “suffer from” the serial bottleneck



Vector Processor Limitations

- Memory (bandwidth) can easily become a bottleneck, especially if
 1. compute/memory operation balance is not maintained
 2. data is not mapped appropriately to memory banks

Vector Machine Organization (CRAY-1)



- CRAY-1
- Russell, “The CRAY-1 computer system,” CACM 1978.
- Scalar and vector modes
- 8 64-element vector registers
- 64 bits per element
- 16 memory banks
- 8 64-bit scalar registers
- 8 24-bit address registers

CRAY X-MP-28 @ ETH (CAB, E Floor)



Cray X-MP-28

Der von Seymour Cray entworfene Supercomputer Cray X-MP besticht durch seinen leicht theatralischen Auftritt. Von 1983 bis 1988 galt der 5,5 Tonnen schwere und bis zu 15 Millionen Dollar teure Vektor-Koloss als schnellster Computer der Welt.

Die Anschaffung des Cray X-MP/28 im Jahr 1988 markiert den Ausgangspunkt für das Engagement der ETH, auch im Bereich des Hochleistungsrechnens vorne mit dabei zu sein.

Beim ausgestellten System handelt es sich lediglich um die Prozessoreinheit. Zusätzlich war noch ein I/O System zum Anschluss von Bandlaufwerken und Festplatten Bestandteil des Rechners.

Für den Betrieb waren an der ETH stets vier Angestellte von Cray Research vor Ort. Zwei für die Wartung der Hardware, zwei für die Programmierung und Administration.

Seit 1991 sind die Supercomputer der ETH Zürich im Swiss National Supercomputing Centre (SCS) im Tessin zuhause. Aktuell ist es wieder ein Cray, der dort für Spitzenleistungen sorgt. «Piz Daint» genannt, gilt der ETH-Supercomputer seit Ende 2013 als schnellster und energieeffizientester Rechner Europas.

Miniaturisierung und explodierende Leistung

Wie rasend schnell sich die Leistungsfähigkeit der Hardware entwickelt hat, zeigt der Vergleich des gelben Riesen mit einem Mini-Computer von heute.

Cray X-MP/28

Zwei parallele Vektorprozessoren mit 118 MHz Systemtakt bringen eine maximale Rechenleistung von 400 MegaFlops. Anschaffungspreis 1988: rund 5 Millionen Franken.

Raspberry Pi 1 model B+

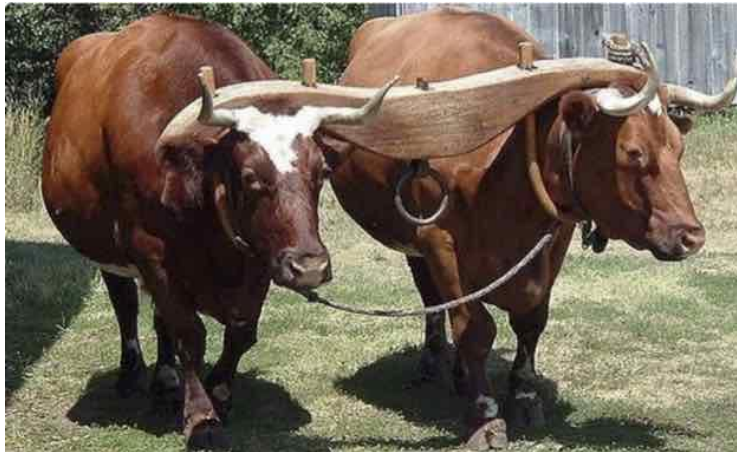
Die gleichen 400 MegaFlops werden heute zum Beispiel von der untenstehenden Einplatinencomputer, ausgestattet mit einem ARM11 Einkernprozessor mit 1000 MHz Systemtakt, erreicht. Anschaffungspreis 2015: 32 Franken.



Seymour Cray, Leader in Supercomputer Design



"If you were plowing a field, which would you rather use: **Two strong oxen** or **1024 chickens**?"



© amityrebecca / Pinterest. <https://www.pinterest.ch/pin/473018767088408061/>



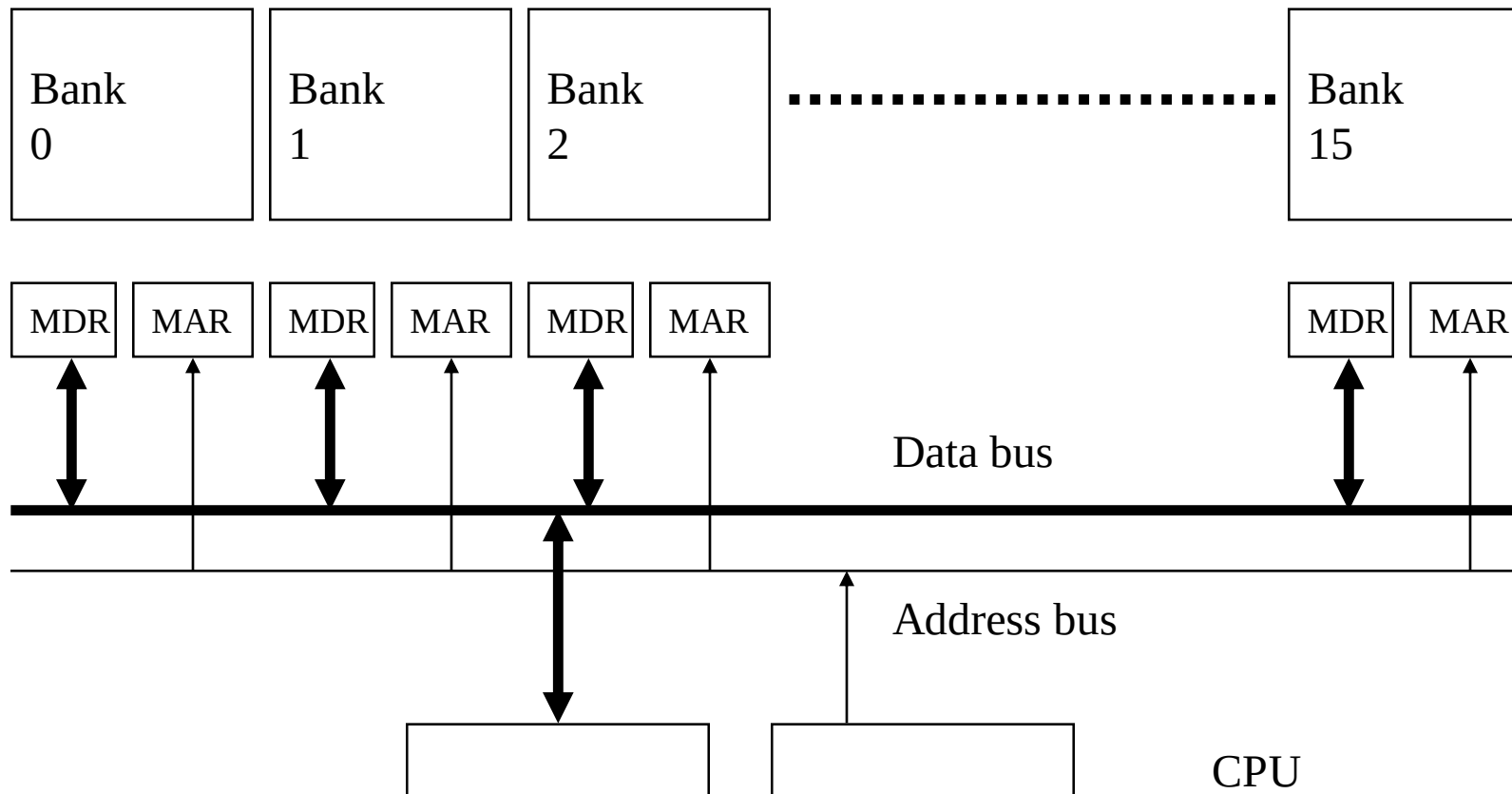
© Scott Sinkler / Corbis. <http://america.aljazeera.com/articles/2015/2/20/the-short-brutal-life-of-male-chickens.html>

Loading/Storing Vectors from/to Memory

- Requires loading/storing multiple elements
- Elements separated from each other by a constant distance (stride)
 - Assume stride = 1 for now
- Elements can be loaded in consecutive cycles if we can start the load of one element per cycle
 - Can sustain a throughput of one element per cycle
- Question: How do we achieve this with a memory that takes **more than 1 cycle to access**?
- Answer: **Bank** the memory; **interleave** the elements across banks

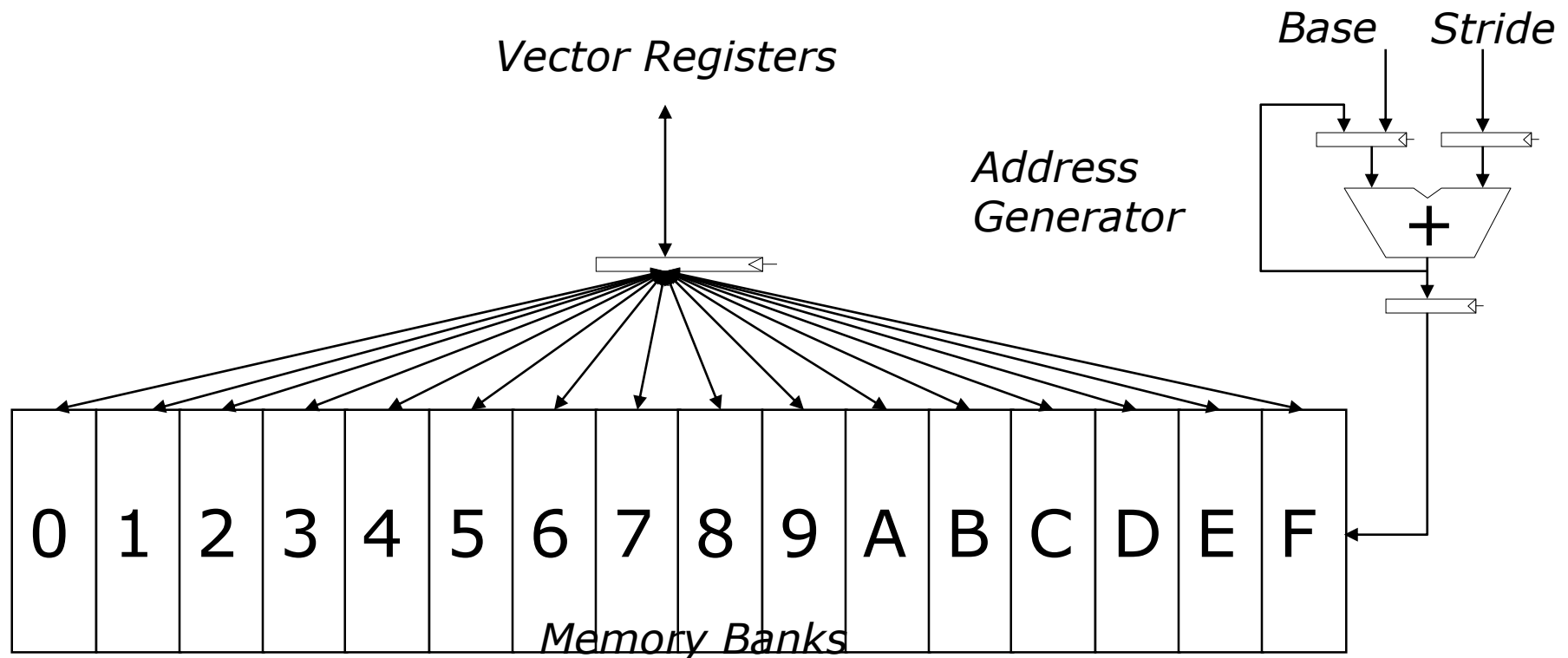
Memory Banking

- Memory is divided into **banks** that can be accessed independently; banks share address and data buses (to reduce memory chip pins)
- Can start and complete one bank access per cycle
- Can sustain N concurrent accesses if all N go to different banks



Vector Memory System

- Next address = Previous address + Stride
- If (stride == 1) && (consecutive elements interleaved across banks) && (number of banks $\geq$ bank latency), then
 - we can sustain 1 element/cycle throughput



Scalar Code Example: Element-Wise Avg.

- For I = 0 to 49
 - ▣ $C[i] = (A[i] + B[i]) / 2$
- Scalar code (instruction and its latency in clock cycles)

MOVI R0 = 50	1	
MOVA R1 = A	1	304 dynamic instructions
MOVA R2 = B	1	
MOVA R3 = C	1	
X: LD R4 = MEM[R1++]	11	;autoincrement addressing
LD R5 = MEM[R2++]	11	
ADD R6 = R4 + R5	4	
SHFR R7 = R6 >> 1	1	
ST MEM[R3++] = R7	11	
DECBNZ R0, X	2	;decrement and branch if NZ

Scalar Code Execution Time (In Order)

- Scalar execution time on an in-order processor with 1 bank
 - First two loads in the loop cannot be pipelined: 2×11 cycles
 - $4 + 50 \times 40 = 2004$ cycles
- Scalar execution time on an in-order processor with 1 bank with 2 memory ports (two different memory accesses can be serviced concurrently) OR 2 banks (where arrays A and B are stored in different banks)
 - First two loads in the loop can be pipelined: $1 + 11$ cycles
 - $4 + 50 \times 30 = 1504$ cycles

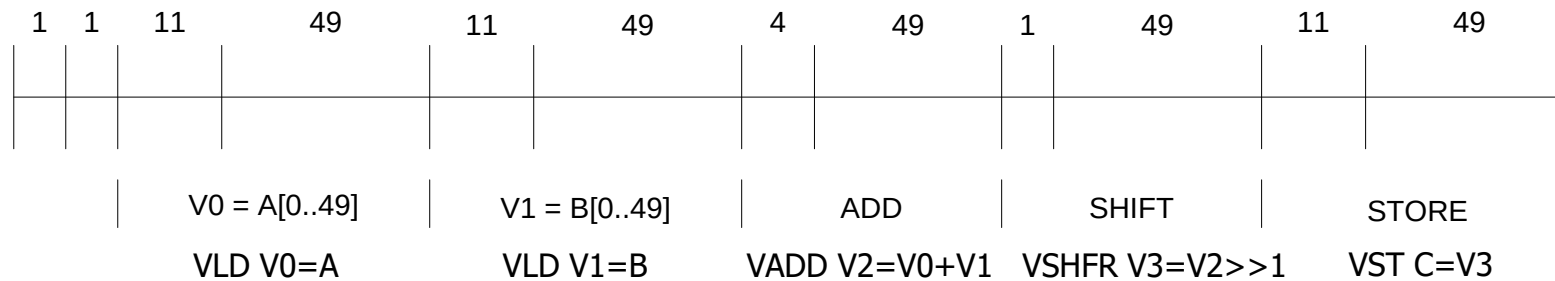
Vectorizable Loops

- A loop is **vectorizable** if each iteration is independent of any other
- For $I = 0$ to 49
 - $C[i] = (A[i] + B[i]) / 2$
- Vectorized loop (each instruction and its latency):

MOVI VLEN = 50	1	7 dynamic instructions
MOVI VSTR = 1	1	
VLD V0 = A	$11 + \text{VLEN} - 1$	
VLD V1 = B	$11 + \text{VLEN} - 1$	
VADD V2 = V0 + V1	$4 + \text{VLEN} - 1$	
VSHFR V3 = V2 >> 1	$1 + \text{VLEN} - 1$	
VST C = V3	$11 + \text{VLEN} - 1$	

Basic Vector Code Performance

- Assume **no chaining** (no vector data forwarding)
 - i.e., output of a vector functional unit cannot be used as the direct input of another
 - **The entire vector register needs to be ready** before any element of it can be used as part of another operation
- 1 memory port (one address generator) per bank
- 16 memory banks (word-interleaved: consecutive elements of an array are stored in consecutive banks)



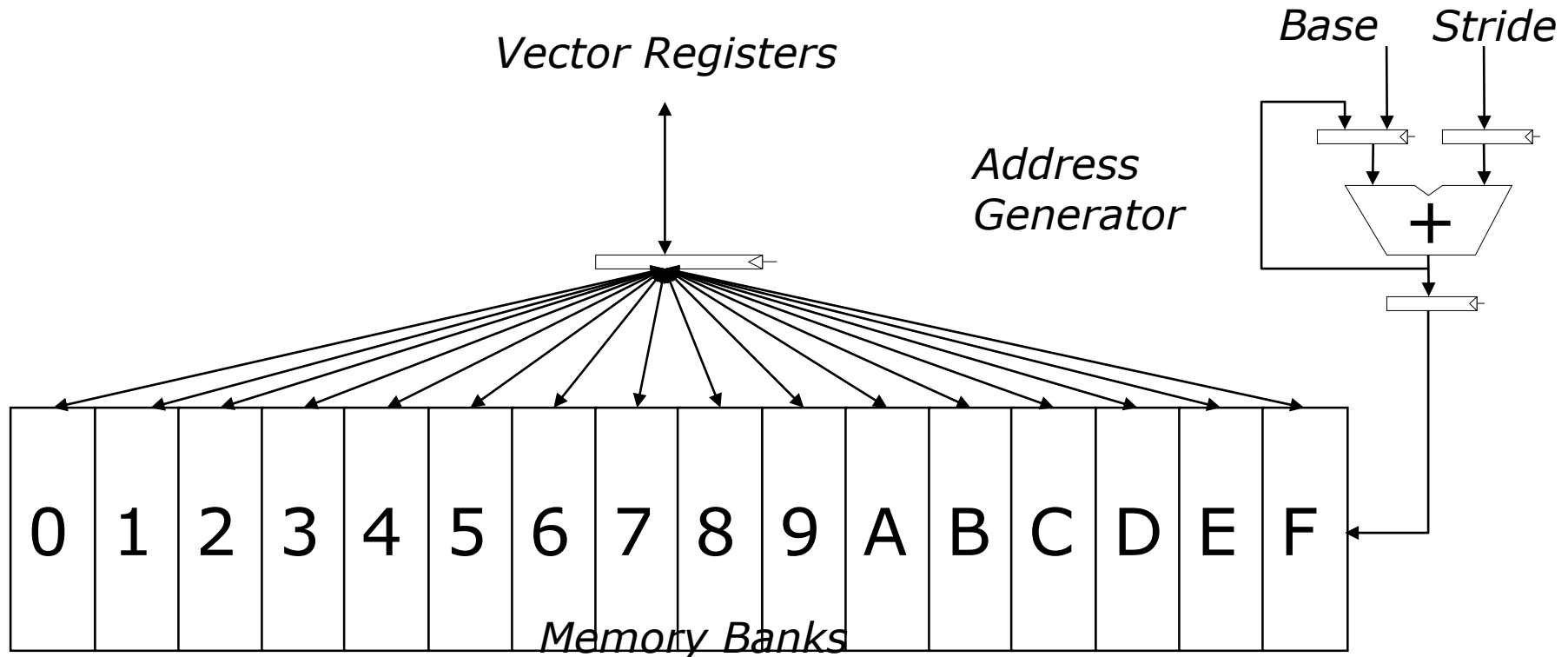
- 285 cycles

Basic Vector Code Performance (II)

- Why 16 banks?
 - 11-cycle memory access latency
 - Having 16 (>11) banks ensures there are enough banks to overlap enough memory operations to cover memory latency
- The above assumes a unit stride (i.e., stride = 1)
 - Correct for our example program
- What if stride > 1 ?
 - How do you ensure we can access 1 element per cycle when memory latency is 11 cycles?

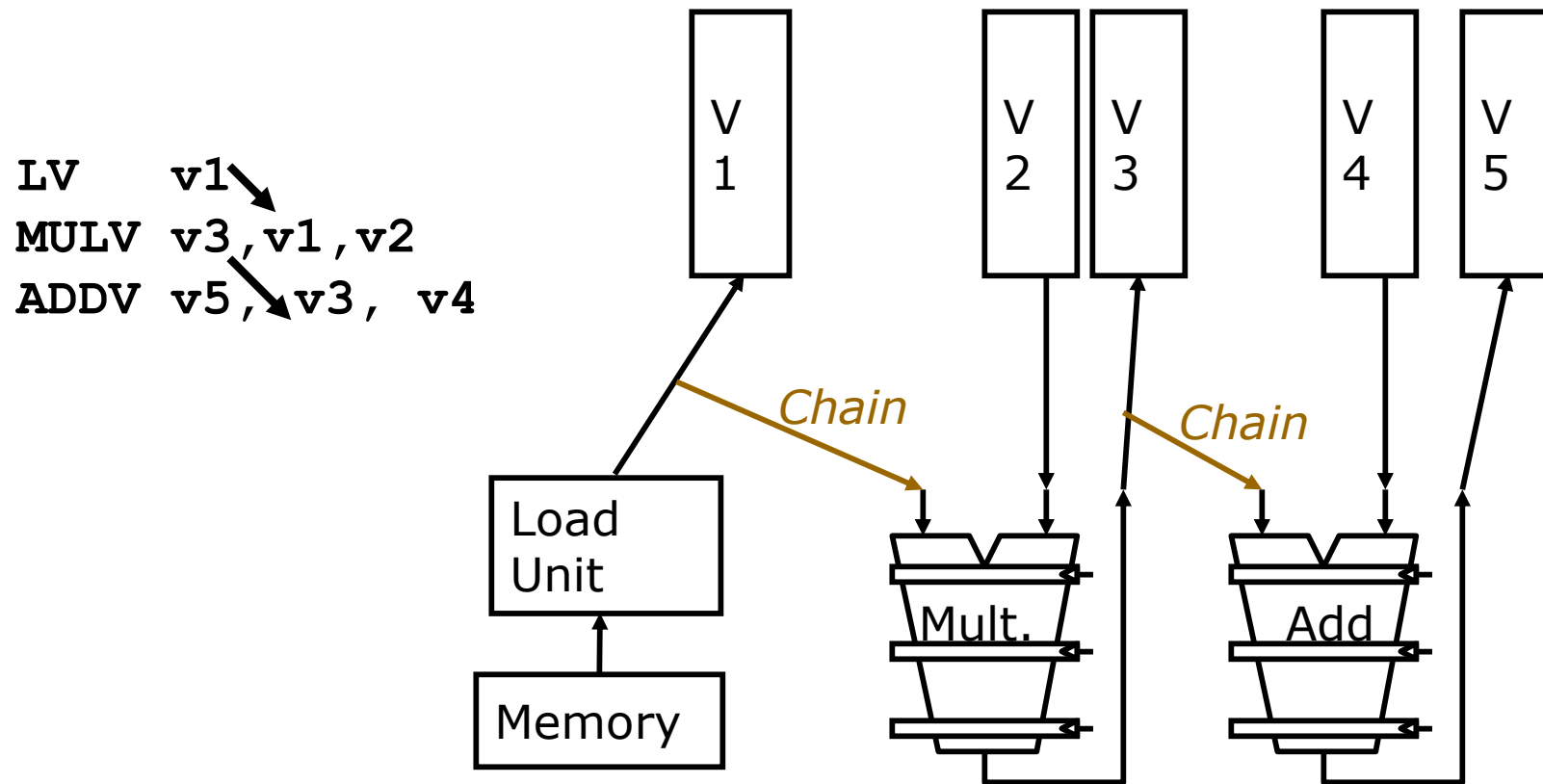
Recall: Vector Memory System

- Next address = Previous address + Stride
- If (stride == 1) && (consecutive elements interleaved across banks) && (number of banks >= bank latency), then
 - we can sustain 1 element/cycle throughput



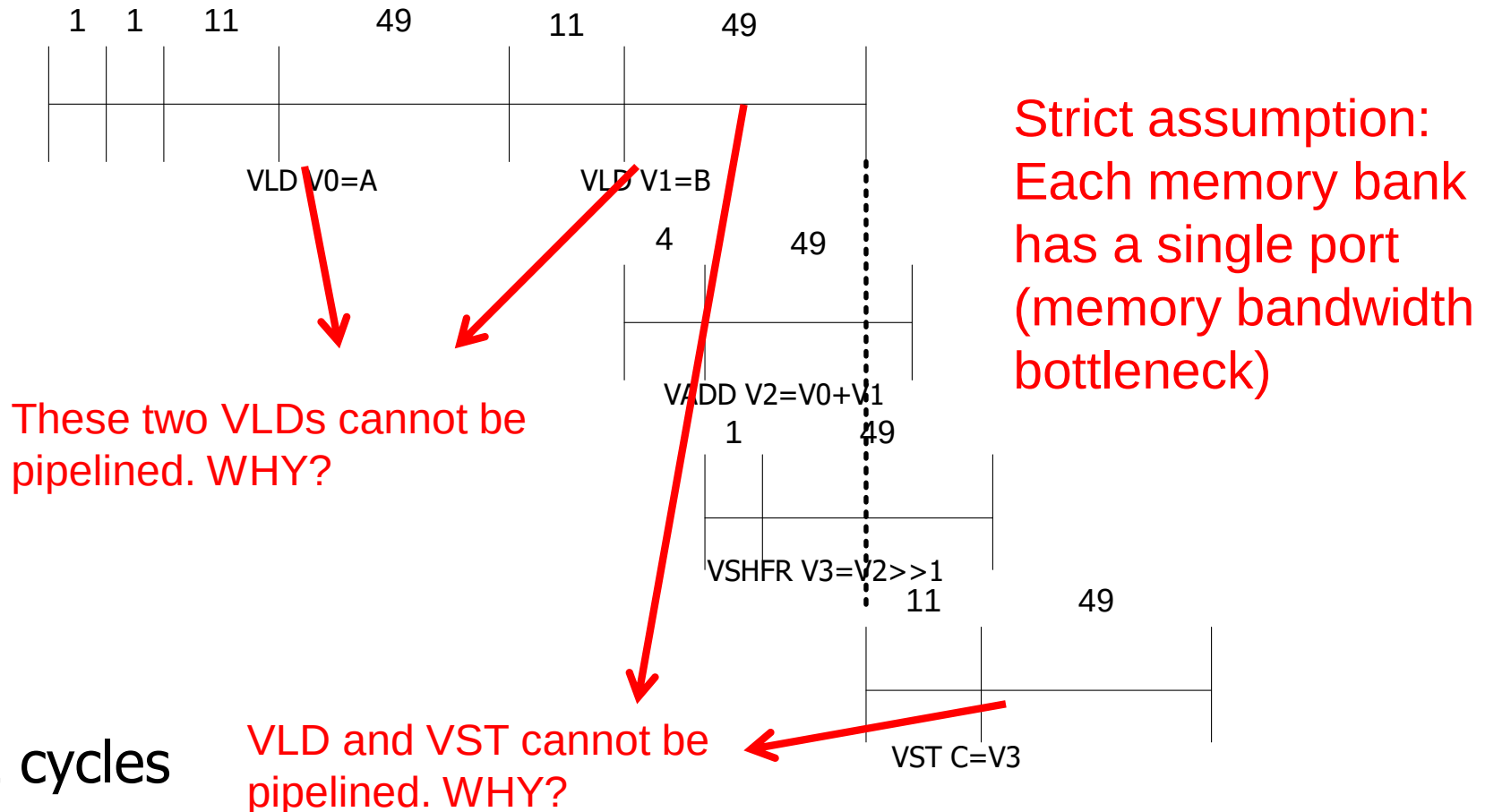
Vector Chaining

- **Vector chaining:** Data forwarding from one vector functional unit to another



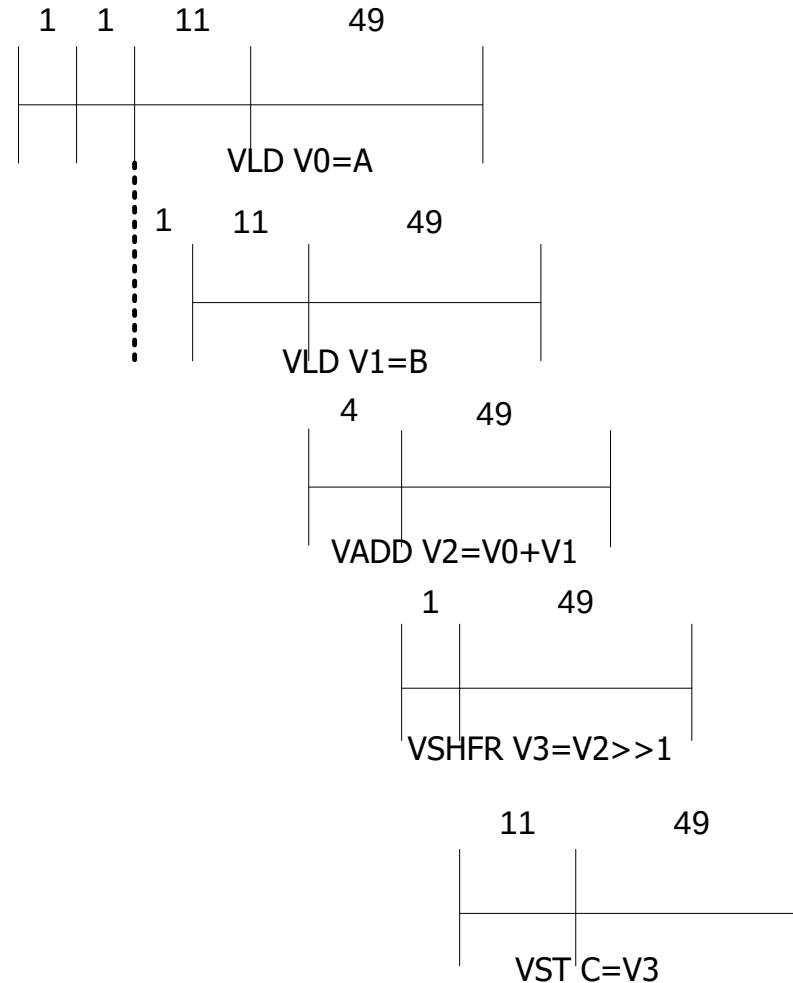
Vector Code Performance - Chaining

- **Vector chaining:** Data forwarding from one vector functional unit to another



Vector Code Performance – Multiple Memory Ports

- Chaining and 2 load ports, 1 store port in each bank



- 79 cycles
- 19X perf. improvement!

Questions (I)

- What if # data elements > # elements in a vector register?
 - Idea: Break loops so that each iteration operates on # elements in a vector register
 - E.g., 527 data elements, 64-element VREGs
 - 8 iterations where $VLEN = 64$
 - 1 iteration where $VLEN = 15$ (need to change value of $VLEN$)
 - Called vector stripmining

(Vector) Stripmining

Surface mining, including **strip mining**, **open-pit mining** and **mountaintop removal mining**, is a broad category of **mining** in which soil and rock overlying the mineral deposit (the **overburden**) are removed, in contrast to **underground mining**, in which the overlying rock is left in place, and the mineral removed through shafts or tunnels.

Surface mining began in the mid-sixteenth century^[1] and is practiced throughout the world, although the majority of surface coal mining occurs in North America.^[2] It gained



Coal strip mine in [Wyoming](#)



Questions (II)

- What if vector data is not stored in a strided fashion in memory? (irregular memory access to a vector)
 - ❑ Idea: Use indirection to combine/pack elements into vector registers
 - ❑ Called scatter/gather operations
 - ❑ Doing so also helps with avoiding useless computation on sparse vectors (i.e., vectors where many elements are 0)

Gather/Scatter Operations

Want to vectorize loops with indirect accesses:

```
for (i=0; i<N; i++)  
    A[i] = B[i] + C[D[i]]
```

Indexed load instruction (*Gather*)

```
LV vD, rD          # Load indices in D vector  
LVI vC, rC, vD      # Load indirect from rC base  
LV vB, rB           # Load B vector  
ADDV.D vA, vB, vC   # Do add  
SV vA, rA           # Store result
```

Gather/Scatter Operations

- Gather/scatter operations often implemented in hardware to handle **sparse vectors (matrices)** or **indirect indexing**
- Vector loads and stores use an index vector which is added to the base register to generate the addresses
- *Scatter* example

Index Vector	Data Vector (to Store)	Stored Vector (in Memory)	
0	3.14	Base+0	3.14
2	6.5	Base+1	X
6	71.2	Base+2	6.5
7	2.71	Base+3	X
		Base+4	X
		Base+5	X
		Base+6	71.2
		Base+7	2.71

Conditional Operations in a Loop

- What if some operations should not be executed on a vector (based on a dynamically-determined condition)?

```
loop:      for (i=0; i<N; i++)  
           if (a[i] != 0) then b[i]=a[i]*b[i]
```

- Idea: **Masked operations**

- VMASK register is a bit mask determining which data element should not be acted upon

VLD V0 = A

VLD V1 = B

VMASK = (V0 != 0)

VMUL V1 = V0 * V1

VST B = V1

- This is **predicated execution**. Execution is *predicated* on mask bit.

Another Example with Masking

```
for (i = 0; i < 64; ++i)
  if (a[i] >= b[i])
    c[i] = a[i]
  else
    c[i] = b[i]
```

A	B	VMASK
1	2	0
2	2	1
3	2	1
4	10	0
-5	-4	0
0	-3	1
6	5	1
-7	-8	1

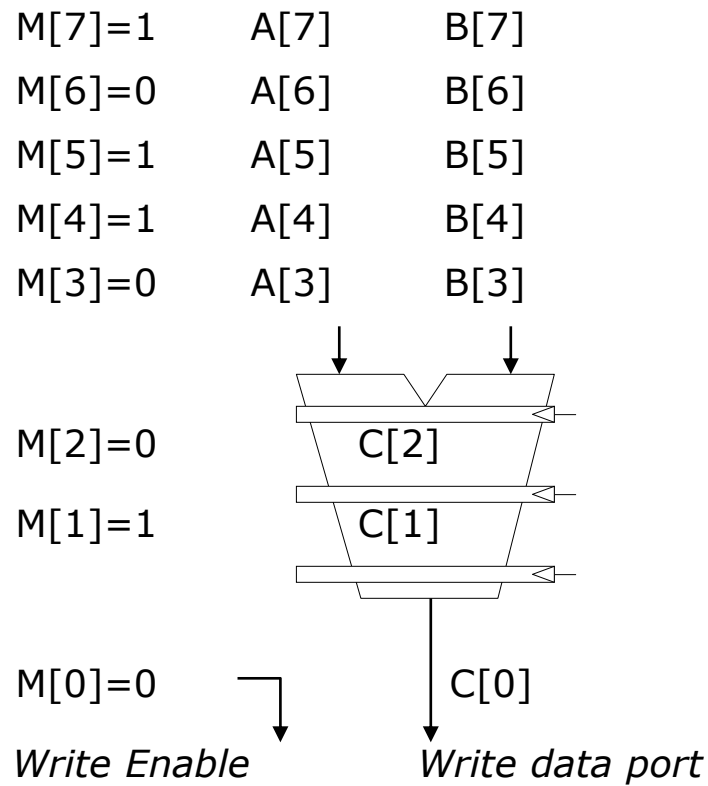
Steps to execute the loop in SIMD code

1. Compare A, B to get VMASK
2. Masked store of A into C
3. Complement VMASK
4. Masked store of B into C

Masked Vector Instructions

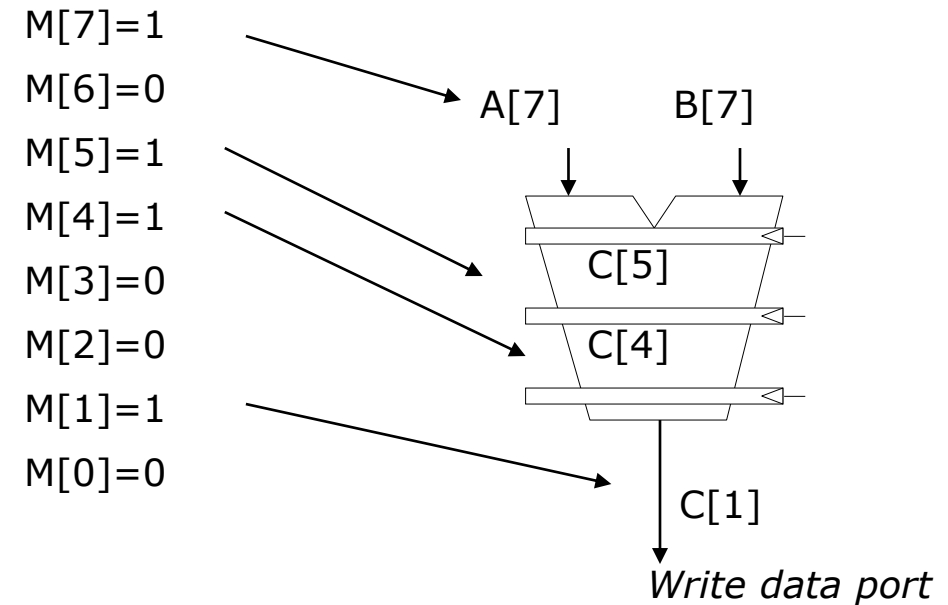
Simple Implementation

- execute all N operations, turn off result writeback according to mask



Density-Time Implementation

- scan mask vector and only execute elements with non-zero masks



Which one is better?

Tradeoffs?

Some Issues

- Stride and bank count

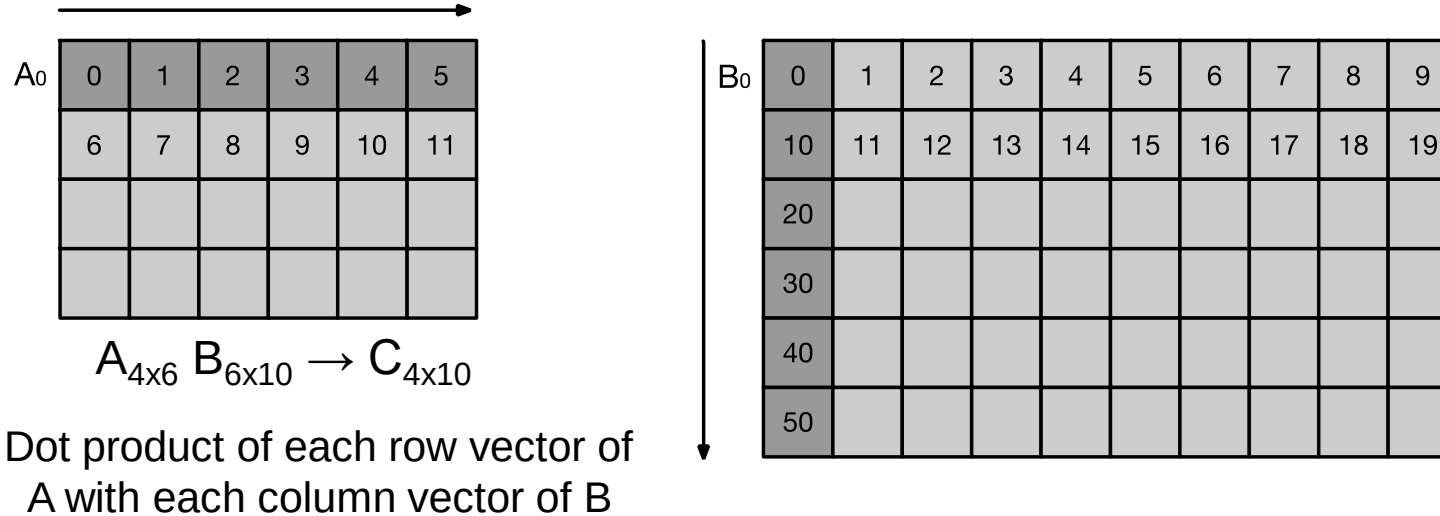
- As long as stride and bank count are *relatively prime* to each other and there are enough banks to cover bank access latency, we can sustain 1 element/cycle throughput

- Storage format of a matrix

- **Row major**: Consecutive elements in a row are laid out consecutively in memory
- **Column major**: Consecutive elements in a column are laid out consecutively in memory
- You need to change the stride when accessing a row versus column

Bank Conflicts in Matrix Multiplication

- A and B matrices, both stored in memory in **row-major order**



- Load A's row 0 into vector register V_1

- ❑ Each time, increment address by **1** to access the next column
- ❑ Accesses have a **stride of 1**

- Load B's column 0 into vector register V_2

- ❑ Each time, increment address by 10
- ❑ Accesses have a **stride of 10**

Different strides can lead to **bank conflicts**

How do we minimize them?

Minimizing Bank Conflicts

- More banks
- More ports in each bank
- Better data layout to match the access pattern
 - Is this always possible?
- Better mapping of address to bank
 - E.g., randomized mapping
 - Rau, “Pseudo-randomly interleaved memory,” ISCA 1991.

Recommended Reading: Minimizing Bank Conflicts

PSEUDO-RANDOMLY INTERLEAVED MEMORY

B. Ramakrishna Rau
Hewlett Packard Laboratories
1501 Page Mill Road
Palo Alto, CA 94303

ABSTRACT

Interleaved memories are often used to provide the high bandwidth needed by multiprocessors and high performance uniprocessors such as vector and VLIW processors. The manner in which memory locations are distributed across the memory modules has a significant influence on whether, and for which types of reference patterns, the full bandwidth of the memory system is achieved. The most common interleaved memory architecture is the sequentially interleaved memory in which successive memory locations are assigned to successive memory modules. Although such an architecture is the simplest to implement and provides good performance with strides that are odd integers, it can degrade badly in the face of even strides, especially strides that are a power of two.

In a pseudo-randomly interleaved memory architecture, memory locations are assigned to the memory modules in some pseudo-random fashion in the hope that those sequences of references, which are likely to occur in practice, will end up being evenly distributed across the memory modules. The notion of polynomial interleaving modulo an irreducible polynomial is introduced as a way of achieving pseudo-random interleaving with certain attractive and provable properties. The theory behind this scheme is developed and the results of simulations are presented.

Keywords: supercomputer memory, parallel memory, interleaved memory, hashed memory, pseudo-random interleaving, memory buffering.

The conventional solution is to provide each processor with a data cache constructed out of SRAM. The problem is maintaining cache coherency, at high request rates, across multiple private caches in a multiprocessor system. The alternative is to use a shared cache if the additional delay incurred in going through the processor-cache interconnect is acceptable. The problem here is that the bandwidth, even with SRAM chips, is inadequate unless some form of interleaving is employed in the cache. So once again, the interleaving scheme used is an issue. Furthermore, data caches are susceptible to problems arising out of the lack of spatial and/or data locality in the data reference pattern of many applications. This phenomenon has been studied and reported elsewhere, e.g., in [4,5]. Since data caches are essential to achieving good performance on scalar computations with little parallelism, the right compromise is to provide a data cache that can be bypassed when referencing data structures with poor locality. This is the solution employed in various recent products such as the Convex C-1 and Intel's i860.

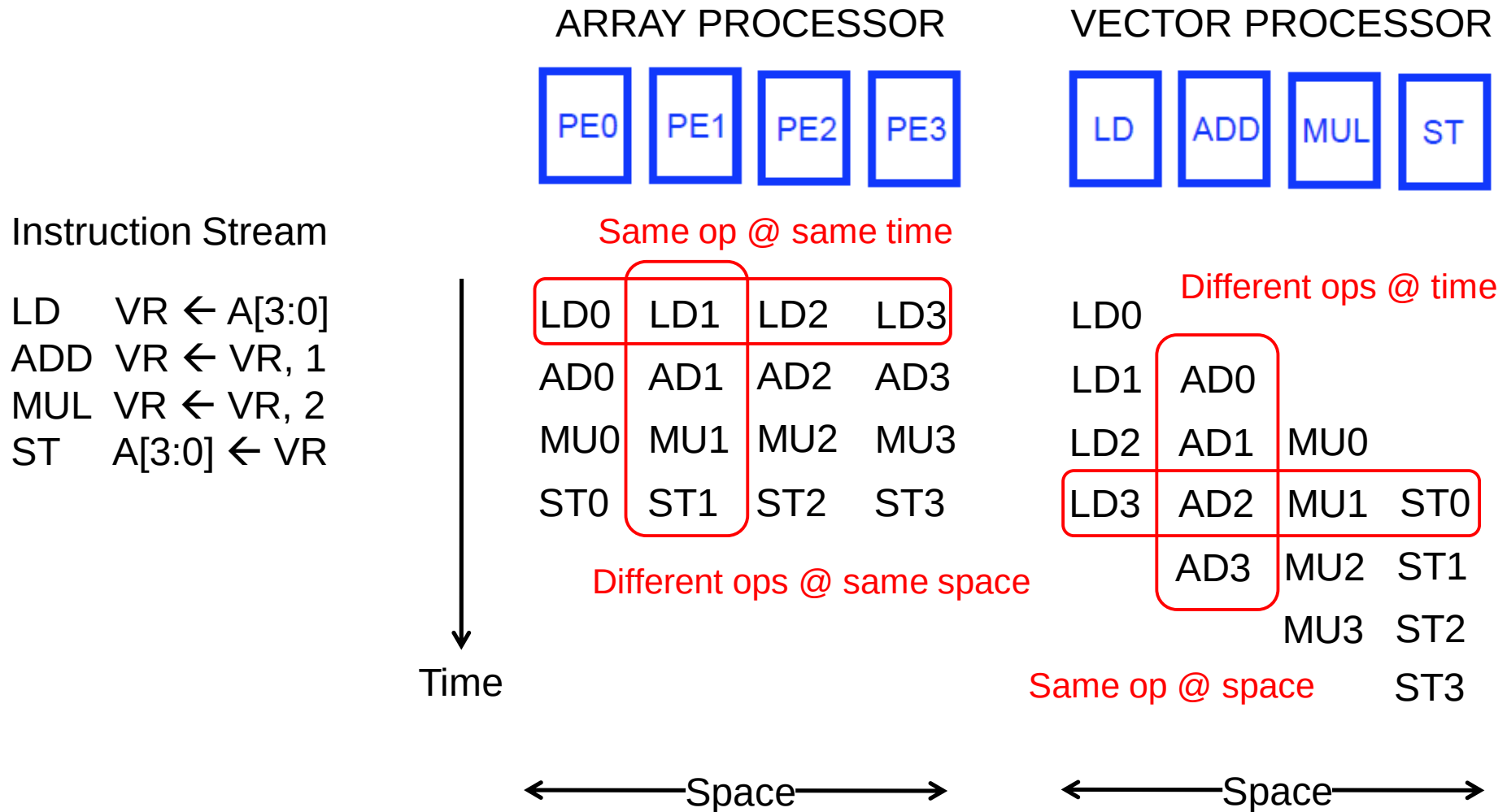
Interleaved memory systems. Whether or not a data cache is present, it is important to provide a memory system with bandwidth to match the processors. This is done by organizing the memory system as multiple memory modules which can operate in parallel. The manner in which memory locations are distributed across the memory modules has a significant influence on whether, and for which types of reference patterns, the full bandwidth of the memory system is achieved.

Engineering and scientific applications include

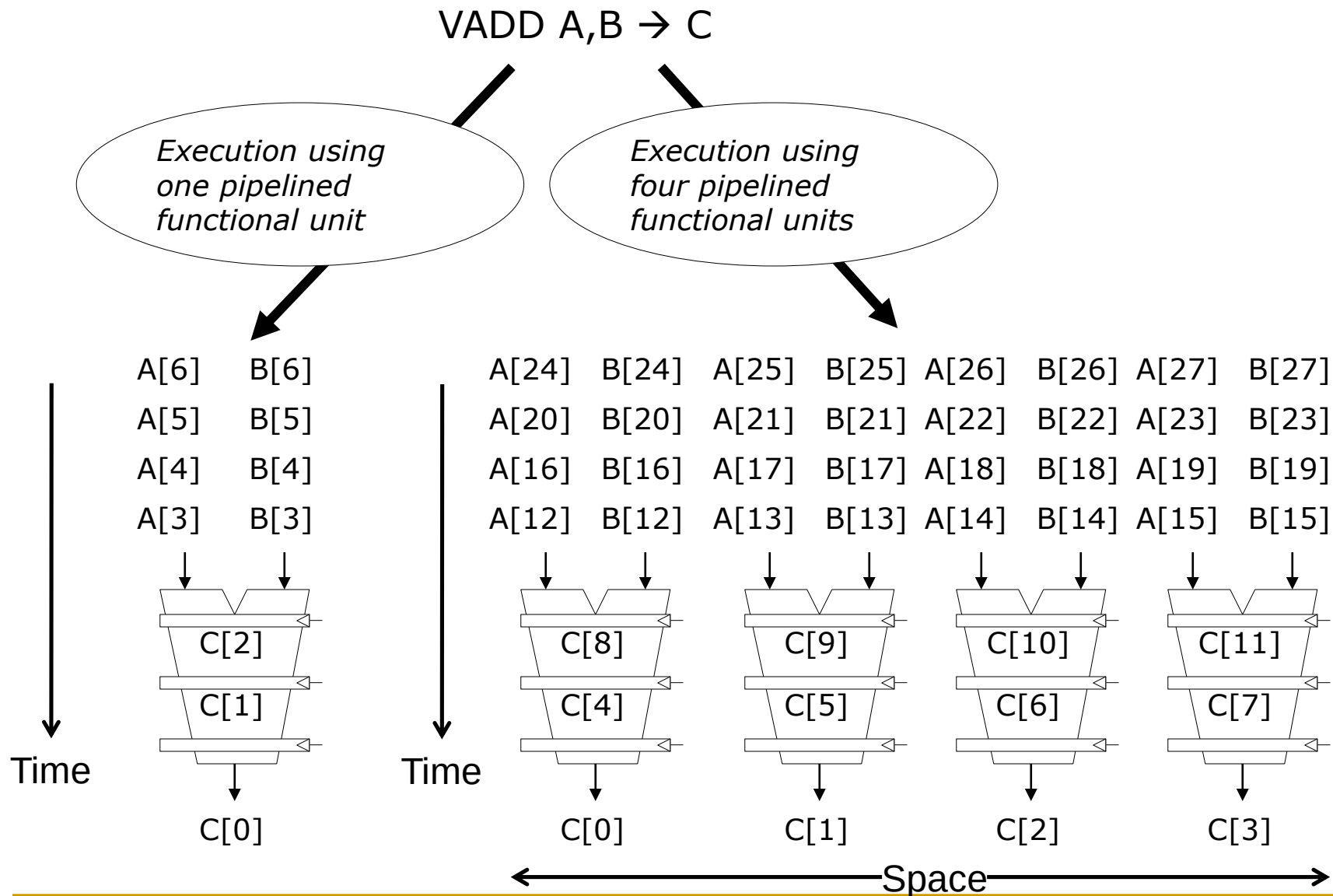
Array vs. Vector Processors, Revisited

- Array vs. vector processor distinction is a “purist’s” distinction
- Most “modern” SIMD processors are a combination of both
 - They exploit data parallelism in both time and space
 - GPUs are a prime example.

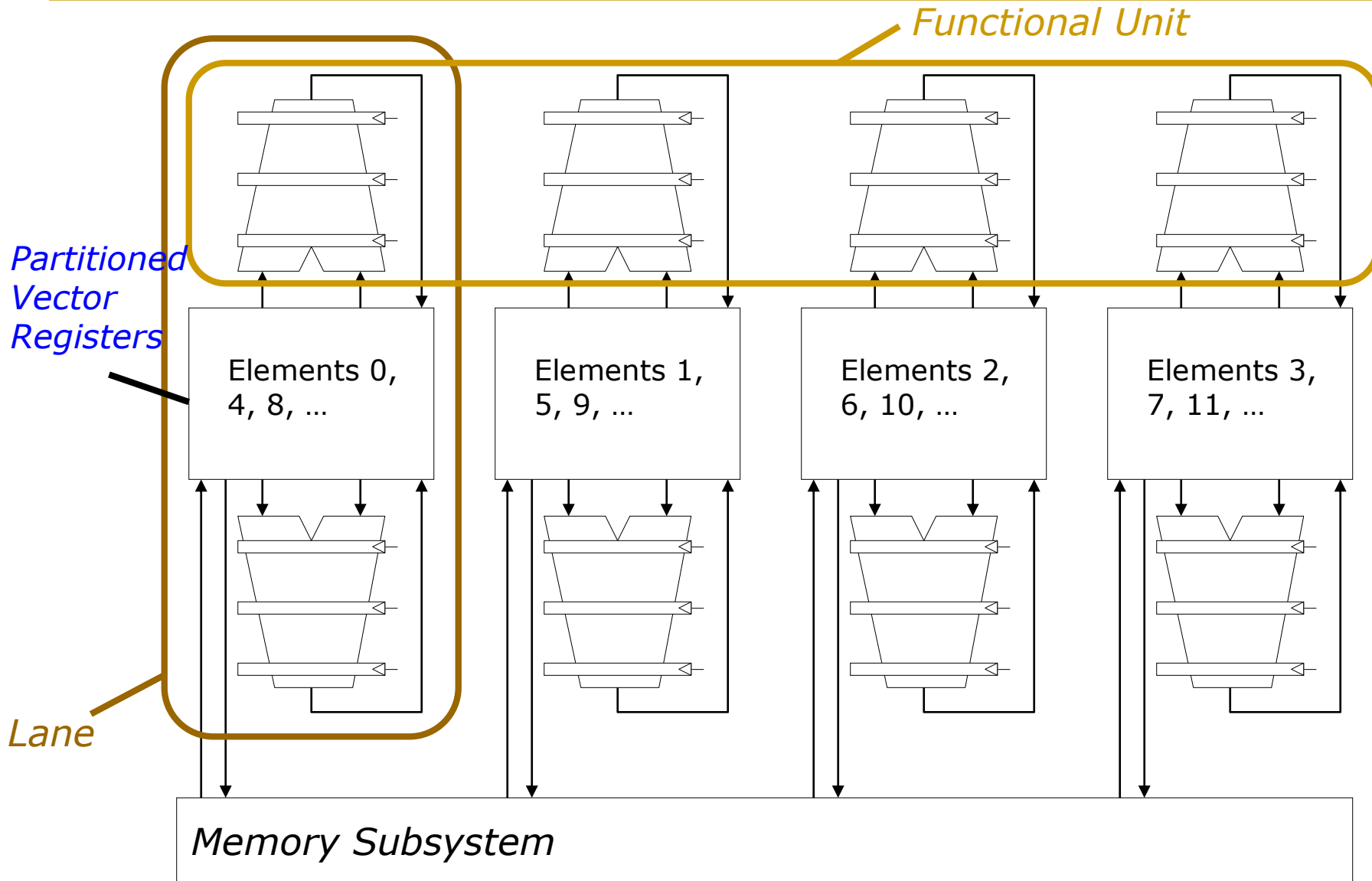
Recall: Array vs. Vector Processors



Vector Instruction Execution



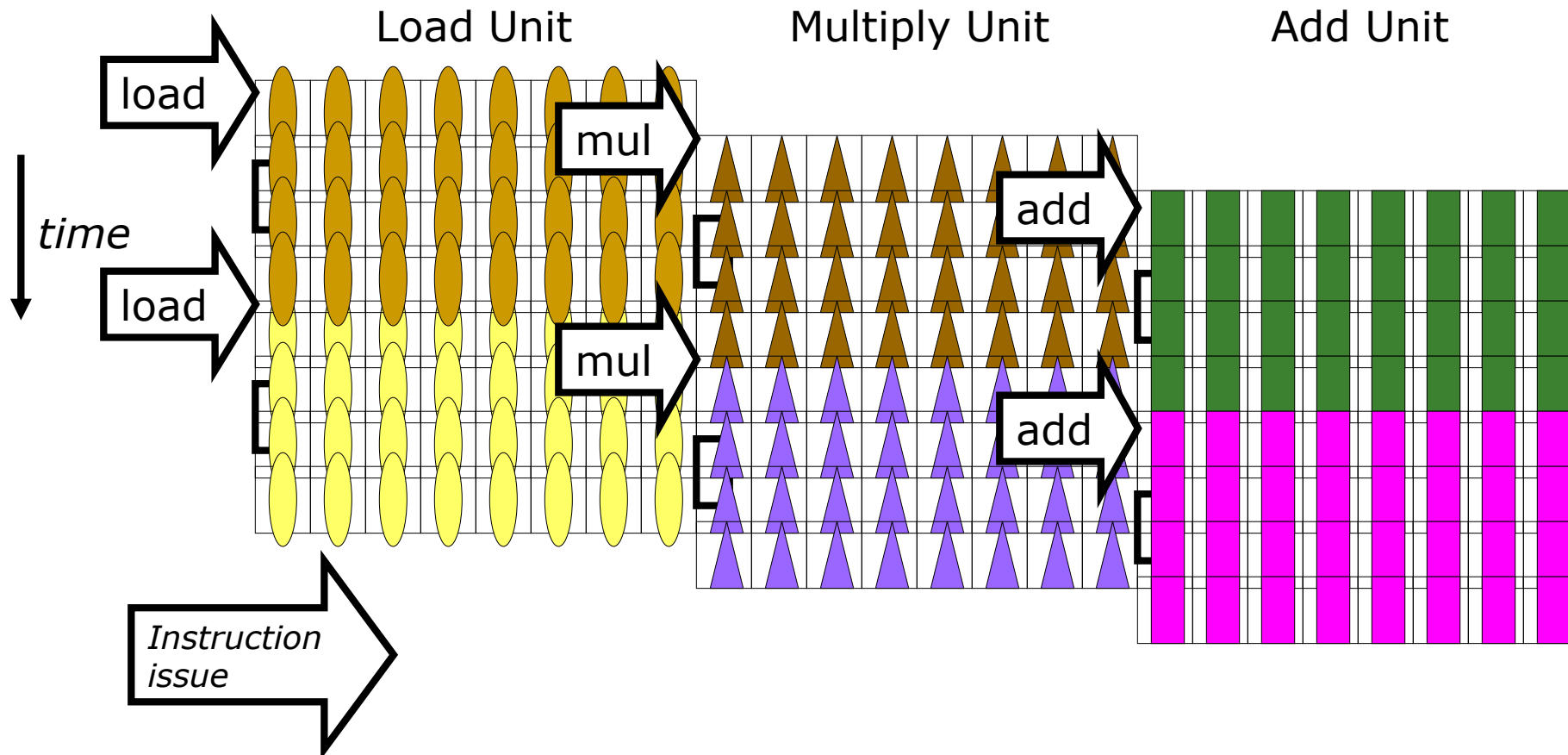
Vector Unit Structure



Vector Instruction Level Parallelism

Can overlap execution of multiple vector instructions

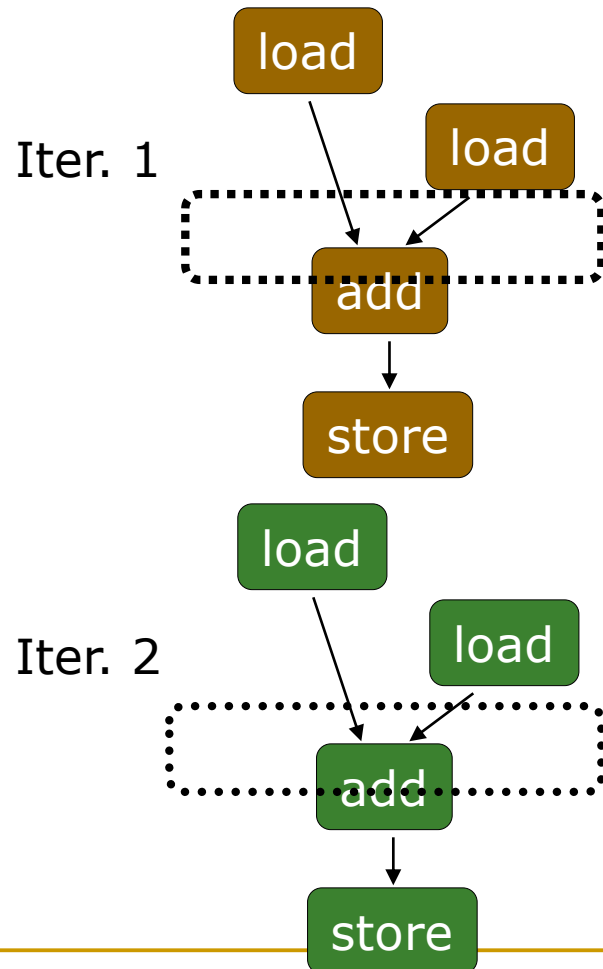
- Example machine has 32 elements per vector register and 8 lanes
- Example with 24 operations/cycle (steady state) while issuing 1 vector instruction/cycle



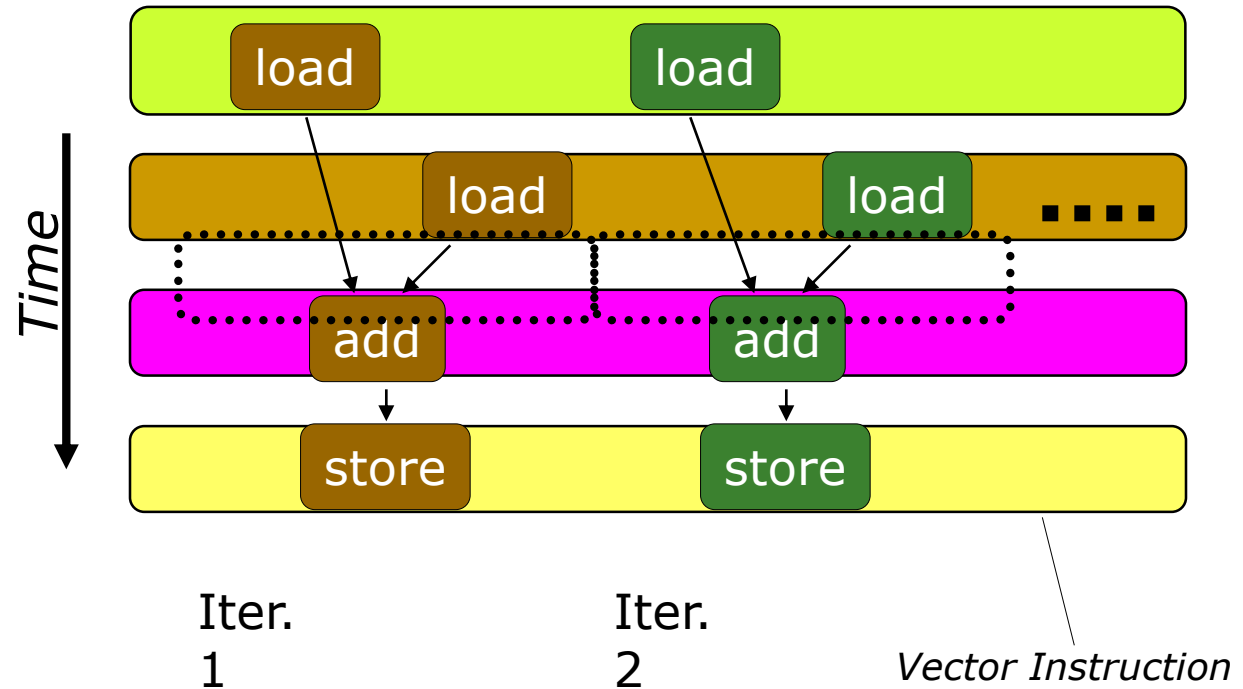
Automatic Code Vectorization

```
for (i=0; i < N; i++)  
  C[i] = A[i] + B[i];
```

Scalar Sequential Code



Vectorized Code



Vectorization is a compile-time reordering of operation sequencing
⇒ requires extensive loop dependence analysis

Vector/SIMD Processing Summary

- Vector/SIMD machines are good at exploiting **regular data-level parallelism**
 - ❑ Same operation performed on many data elements
 - ❑ Improve performance, simplify design (no intra-vector dependencies)
- **Performance improvement limited by vectorizability** of code
 - ❑ Scalar operations limit vector machine performance
 - ❑ Remember **Amdahl's Law**
 - ❑ CRAY-1 was the fastest SCALAR machine at its time!
- Many existing ISAs include (vector-like) SIMD operations
 - ❑ Intel MMX/SSEn/AVX, PowerPC AltiVec, ARM Advanced SIMD

GPUs are SIMD Engines Underneath

- The **instruction pipeline operates like a SIMD pipeline** (e.g., an array processor)
- However, the **programming is done using threads**, NOT SIMD instructions
- To understand this, let's go back to our parallelizable code example
- But, before that, let's distinguish between
 - **Programming Model (Software)**
 - vs.
 - **Execution Model (Hardware)**

Programming Model vs. Hardware Execution Model

- Programming Model refers to **how the programmer expresses the code**
 - E.g., Sequential (von Neumann), Data Parallel (SIMD), Dataflow, Multi-threaded (MIMD, SPMD), ...
- Execution Model refers to **how the hardware executes the code underneath**
 - E.g., Out-of-order execution, Vector processor, Array processor, Dataflow processor, Multiprocessor, Multithreaded processor, ...
- **Execution Model can be very different from the Programming Model**
 - E.g., von Neumann model implemented by an OoO processor
 - E.g., SPMD model implemented by a SIMD processor (a GPU)

Shared Memory Parallel Computers

- One of the most common parallel architectures using moderate number of processors (4 to 32) is a shared memory multiprocessor [Culler, Singh and Gupta, 1999].
- This architecture provides a global address space for writing parallel programs. Each processor has a private cache. The processors are connected to the main memory either using a shared bus or using an interconnection network.
- In both these cases the average access time to the main memory from any processor is same. Thus, this architecture is also known as **Symmetric Multiprocessor** abbreviated SMP.
- This architecture is popular as it is easy to program it due to the availability of globally addressed main memory where the program and all data are stored. A shared bus parallel machine is also inexpensive and easy to expand by adding more processors.

